

Hochschule Aalen

Technology and Economy

Project Report

Course of Studies: Machine Learning and Data Analytics

Using Mixture of Experts (MoE) to Learn New Data Without Losing Old Knowledge

by

Gajera Yash Hasmukhbhai

3015550

Supervising Professor: Prof. Dr. [First Examiner]

Submission Date: 15. August 2026

Statutory Declaration

I, Gajera Yash Hasmukhbhai, hereby declare that I have written the present work truthfully and independently by myself. Furthermore, I assure that I have used no other than the specified sources and aids, that I have marked all quotations that I have used from other sources, and that the work in the same or similar form was not yet part of a study or examination.

As aids, the programming language Python with the libraries PyTorch, torchvision, NumPy and Matplotlib were used for the implementation and the experiments, as well as an AI based assistant for support in wording and code drafting. All generated content was checked, tested and evaluated by me, and I take full responsibility for the content of this work.

Aalen, 15. August 2026

Signature (Student)

Abstract

In many practical applications, a machine learning model does not receive all training data at once. New data arrives step by step, and a model that is updated on new data usually forgets what it has learned before. This problem is called catastrophic forgetting. The goal of this project is to investigate whether a Mixture of Experts model, in which a small gating network routes every input to a few specialised sub networks, forgets less than a normal neural network when tasks are learned one after another.

For this purpose, a complete experimental framework was implemented in Python and PyTorch. Two standard benchmarks were built from the MNIST data set: SplitMNIST with five two class tasks and PermutedMNIST with five random pixel permutations. A naive multi layer perceptron, the same network with Elastic Weight Consolidation, a wider network with more parameters, and the proposed Mixture of Experts model were trained sequentially and compared with the metrics average accuracy, average forgetting and backward transfer. Every experiment was repeated with three random seeds.

The results show that the Mixture of Experts model forgets clearly less than the naive baseline on SplitMNIST. On PermutedMNIST, the first version of the model performed worse than expected. A detailed analysis showed two reasons: the load balancing loss forces all tasks to share all experts, and more importantly, the task specific information of PermutedMNIST is stored in the shared input layer, which the router cannot protect. A second architecture with routing directly at the input solves this problem and reaches the best knowledge retention of all compared approaches on both benchmarks. An analysis of the gating network confirms that different tasks are routed to different experts, which explains the reduced forgetting.

Contents

Statutory Declaration	i
Abstract	ii
List of Abbreviations	ix
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement and Delimitation	2
1.3 Objective of the Work	3
1.4 Approach	3
2 Fundamentals	5
2.1 Feed Forward Neural Networks	5
2.2 Continual Learning	6
2.2.1 Problem Definition	6
2.2.2 The Three Continual Learning Scenarios	6
2.2.3 The Stability Plasticity Dilemma	7
2.3 Catastrophic Forgetting	7
2.4 Metrics for Continual Learning	8
2.5 Mixture of Experts	8
2.5.1 Historical Roots	8
2.5.2 Sparsely Gated Mixture of Experts	9
2.5.3 Load Balancing	9
2.5.4 Position of the MoE Layer in the Network	10
2.5.5 Why MoE Is Attractive for Continual Learning	10
3 Problem Analysis	12
3.1 Decomposition of the Problem	12

3.2	Analysis of Existing Solution Approaches	12
3.2.1	Regularisation Based Methods	13
3.2.2	Replay Based Methods	13
3.2.3	Architecture Based Methods	13
3.2.4	Comparison of the Method Families	14
3.2.5	Conclusion for this Project	14
3.3	Requirements Analysis	15
4	Solution Concept	17
4.1	Architectural Overview	17
4.2	Concept of the MoE Model	17
4.3	Concept of the Router	18
4.4	Concept of the Training Procedure	19
5	Implementation	21
5.1	Development Tools	21
5.2	Module Structure	21
5.3	Benchmark Construction	22
5.3.1	SplitMNIST	22
5.3.2	PermutedMNIST	22
5.4	Model Implementations	22
5.5	Training Strategies	23
5.5.1	Naive Fine Tuning	23
5.5.2	Elastic Weight Consolidation	23
5.5.3	MoE Training	24
5.6	Experiment Drivers and Result Storage	24
6	Commissioning	25
6.1	Preparation of the Environment	25
6.2	Execution of the Experiments	25
6.3	Reproducibility Measures	26

7	Evaluation	27
7.1	Experimental Setup	27
7.2	Results on SplitMNIST	28
7.3	Results on PermutedMNIST	30
7.4	Analysis of the PermutedMNIST Failure	31
7.5	Architecture Variant with Input Level Routing	32
7.6	Router Analysis	33
7.7	Per Task Analysis	35
7.8	Discussion	36
8	Summary and Outlook	38
8.1	Achieved Results	38
8.2	Outlook	39
8.2.1	Extensibility of the Results	39
8.2.2	Transferability of the Results	39
	Bibliography	40
A	Source Code	43
A.1	Benchmark Construction (<code>datasets.py</code>)	43
A.2	Models (<code>models.py</code>)	46
A.3	Strategies and Metrics (<code>continual.py</code>)	51
B	Experiment Driver and Logs	57
B.1	Experiment Driver (<code>run_experiments.py</code>)	57
B.2	Experiment Log	59

List of Figures

1.1. Overview of the approach of this project.	3
4.1. Layered concept of the experimental system. The data flows from the benchmark layer through the model and strategy layers to the evaluation layer, which produces the accuracy matrix, the scalar metrics and the router profiles.	17
4.2. Concept of the MoE model. A shared trunk feeds eight parallel experts. The router (dashed arrows) selects the top 2 experts per sample (highlighted); their outputs are combined as a gate weighted sum and passed to the output head of the current task.	18
7.1. Average accuracy (left) and average forgetting (right) of all compared approaches on both benchmarks (mean over three seeds, error bars show the standard deviation).	28
7.2. Mean accuracy matrix R on SplitMNIST. Entry (i, j) is the test accuracy on task j after training task i ; a column that stays bright towards the bottom indicates good retention of that task.	29
7.3. Development of the per task test accuracy on SplitMNIST over the five training stages (mean of three seeds).	29
7.4. Mean accuracy matrix R on PermutedMNIST.	30
7.5. Development of the per task test accuracy on PermutedMNIST over the five training stages (mean of three seeds).	30
7.6. Router profile of the hidden level MoE on SplitMNIST: mean gate probability of every expert for every task (mean over three seeds).	33
7.7. Router profile of the hidden level MoE on PermutedMNIST with the main setting. All tasks concentrate on the same two experts E2 and E5, which explains the strong forgetting on this benchmark.	34
7.8. Router profile of the input level MoE on PermutedMNIST. The tasks are distributed over clearly different expert subsets.	34
7.9. Cosine similarity between the per task routing distributions. Off diagonal values close to 1 mean that two tasks use the experts in nearly the same way, which indicates a high interference risk; smaller values indicate task specific routing.	35

List of Tables

3.1. Comparison of the three continual learning method families.	14
4.1. Sequential continual training protocol used for all approaches.	20
5.1. Trainable parameters of the compared architectures (with five output heads on SplitMNIST).	22
7.1. Mean runtime of one continual run per approach on the CPU (mean over three seeds).	28
7.2. Results on SplitMNIST (mean $\pm$ standard deviation over three seeds; best value per column in bold).	29
7.3. Results on PermutedMNIST (mean $\pm$ standard deviation over three seeds; best value per column in bold).	30
7.4. Ablation of the load-balancing coefficient α on PermutedMNIST (mean $\pm$ std over three seeds).	31
7.5. Input-level MoE compared to all approaches on both benchmarks (mean $\pm$ std over three seeds).	32
7.6. Final test accuracy per task after the complete task sequence (mean over three seeds).	36

Listings

5.1 Forward pass of the sparsely gated MoE layer (excerpt from <code>models.py</code>). .	23
6.1 Installation and execution of the experiment suite.	25
6.2 Example lines of the experiment log.	25
../moe_continual_learning/src/datasets.py	43
../moe_continual_learning/src/models.py	46
../moe_continual_learning/src/continual.py	51
../moe_continual_learning/src/run_experiments.py	57
../moe_continual_learning/results/experiment_log2.txt	59

List of Abbreviations

AI	Artificial Intelligence
BWT	Backward Transfer
CL	Continual Learning
CPU	Central Processing Unit
EWC	Elastic Weight Consolidation
LwF	Learning without Forgetting
MAS	Memory Aware Synapses
MLP	Multi Layer Perceptron
MNIST	Modified National Institute of Standards and Technology database
MoE	Mixture of Experts
ReLU	Rectified Linear Unit
SI	Synaptic Intelligence
SGD	Stochastic Gradient Descent

1. Introduction

The introduction should awaken the reader's interest in the contents of this report, point out the treated problems, and describe the concepts that were developed for their solution. For this purpose, this chapter first explains the motivation of the project in Section 1.1, then defines and delimits the problem in Section 1.2, fixes the objective of the work in Section 1.3, and finally presents the chosen approach in Section 1.4.

1.1. Motivation

In classical machine learning, it is assumed that the complete training data set is available from the beginning and that all samples are independent and identically distributed. Under this assumption, modern deep neural networks achieve impressive results in image classification, speech recognition and many other areas. In realistic applications, however, this assumption is often not fulfilled. Data arrives step by step: a medical diagnosis system sees new diseases, a recommendation system sees new user groups, and a self driving car sees new traffic situations, weather conditions or regions. In all these cases, the model has to be updated continuously, ideally without storing and reprocessing all historical data again.

A simple update strategy, namely to continue the training of the network on the new data, fails in a drastic and well documented way. The network adapts its weights to the new task and thereby overwrites exactly those weight configurations that stored the solution of the earlier tasks. After only a few update steps, the performance on the old data breaks down, although the architecture and the capacity of the network have not changed. This phenomenon is known as catastrophic forgetting or catastrophic interference and has been studied in neural networks since the late 1980s [1, 2]. It is still seen as one of the central obstacles on the way to adaptive, lifelong learning systems [3, 4].

The research field of continual learning, also called lifelong or incremental learning, develops methods against this problem. Roughly speaking, three families of approaches exist. Regularisation based methods such as Elastic Weight Consolidation [5] punish changes to weights that were important for earlier tasks. Replay based methods store a small memory of old data and mix it into the training of new tasks [6, 7]. Architecture based methods change the structure of the network itself so that new knowledge can be stored in previously unused components [8]. The present project belongs to the third family.

The starting point of this work is the Mixture of Experts architecture. Instead of one single large network, such a model consists of several smaller sub networks, the so called experts, and a learned gating network, also called router, that decides for every input

which experts should process it [9, 10]. The central hypothesis of this project is that this structure is naturally suited for continual learning: if the router learns to send the data of a new task to a different subset of experts than the data of earlier tasks, then the gradient updates for the new task mainly change those experts, and the knowledge stored in the remaining experts stays mostly untouched. In other words, the model can continue learning without overwriting, because the capacity of the model is partitioned instead of being shared completely.

For me as a student, this topic is attractive because it connects two important and modern topics of machine learning in a very practical way. On the one hand, continual learning is a requirement for almost every machine learning system that is used over a long time. On the other hand, Mixture of Experts models are currently one of the most important building blocks of very large language models [11], and it is an interesting question whether their routing mechanism has useful properties beyond pure scaling.

1.2. Problem Statement and Delimitation

A normal deep neural network works well when all training data is presented at the same time. When the data arrives one task after another, the network loses its old abilities quickly. The concrete problem of this project is therefore to build and evaluate a system that fulfils three requirements at the same time. First, plasticity: the system must be able to learn a new task fast and with high accuracy, using only the data of the current task. Second, stability: after a sequence of tasks, the system must still reach a high accuracy on the earlier tasks, without being trained on their data again. Third, efficiency: the additional computational cost and the additional memory compared to a normal network must stay moderate.

These three requirements are in conflict with each other, which is known as the stability plasticity dilemma [12]. A completely stable model cannot learn anything new, and a completely plastic model forgets everything old. The problem of this project is to find out whether a Mixture of Experts architecture can offer a better compromise than a monolithic network.

To keep the problem tractable, the project is delimited in the following way. The investigation is restricted to two established benchmarks that are built from the MNIST data set [13]: SplitMNIST for the task incremental scenario and PermutedMNIST for the domain incremental scenario [14]. The harder class incremental scenario, in which the model also has to infer the task identity of a test sample, is not part of this work. The investigated networks are feed forward networks of moderate size; convolutional architectures and larger image data sets are excluded. Replay methods, which store old data, are deliberately not used,

because the goal is to find out how far a purely architectural mechanism can go without any stored data. Extensions in these directions are discussed as outlook in Chapter 8.

1.3. Objective of the Work

The objective of this work is to build and test a Mixture of Experts model for continual learning and to find out whether it forgets less than a normal neural network. Following the project proposal, this objective is split into five concrete goals:

- G1.** Study why normal neural networks forget old data so quickly, and summarise the theoretical background of continual learning and Mixture of Experts models.
- G2.** Design and implement a Mixture of Experts model with a trainable router that can be trained step by step on a sequence of tasks.
- G3.** Train this model on two different benchmarks that present the data one task after another.
- G4.** Compare the model with simple neural networks and with an established regularisation method to show how much it forgets in comparison.
- G5.** Examine how the router works and how its behaviour contributes to keeping previously learned knowledge.

The work is considered successful if all five goals are reached with reproducible experiments and if the comparison is based on the standard metrics of the field.

1.4. Approach

After problem statement and objective have fixed the starting point and the end point of the work, this section presents the path that was taken. The individual solution steps and their connection are shown in Figure 1.1.

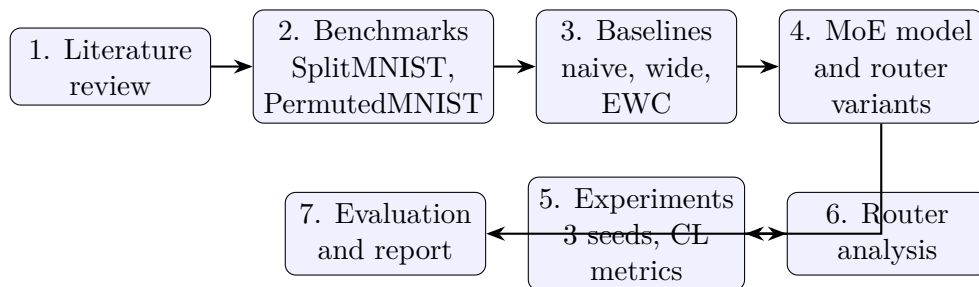


Figure 1.1.: Overview of the approach of this project.

In the first step, the literature on catastrophic forgetting, continual learning and Mixture of Experts models was studied, and the theoretical foundation was worked out. This foundation is presented in Chapter 2. In the second step, the two benchmarks were constructed and the problem was analysed in detail; this analysis and the resulting requirements are documented in Chapter 3. In the third and fourth step, the baseline models and the Mixture of Experts model with its router were designed, which is the content of Chapter 4. The concrete implementation in Python and PyTorch is described in Chapter 5. Chapter 6 explains how the implemented programs were brought into operation and how the experiment suite was executed. Chapter 7 evaluates the results of all experiments, including a detailed router analysis and an architecture variant that resulted from the findings. Chapter 8 finally summarises the achieved results and gives an outlook on possible extensions.

2. Fundamentals

This chapter presents the theoretical knowledge that is needed for the solution of the problem. Only contents that are used later in the report are included. Section 2.1 revisits feed forward neural networks and their training. Section 2.2 formalises continual learning and its standard scenarios. Section 2.3 explains catastrophic forgetting in more detail, and Section 2.4 defines the metrics that are used to measure it. Section 2.5 introduces the Mixture of Experts architecture and its gating mechanism.

2.1. Feed Forward Neural Networks

An artificial neural network is a parametric function $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^c$ that is built from layers of simple computational units. A fully connected layer computes

$$h^{(l)} = \phi(W^{(l)}h^{(l-1)} + b^{(l)}), \quad (2.1)$$

where $h^{(l-1)} \in \mathbb{R}^{d_{l-1}}$ is the input of the layer, $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ is the weight matrix, $b^{(l)} \in \mathbb{R}^{d_l}$ is the bias vector, and ϕ is a non linear activation function. Stacking several such layers gives a multi layer perceptron (MLP). In this project, the rectified linear unit $\phi(z) = \max(0, z)$, short ReLU, is used for the hidden layers, because it is simple, fast and reduces the vanishing gradient problem [15].

For a classification problem with c classes, the last layer produces a vector of logits $z \in \mathbb{R}^c$, which is converted into a probability distribution by the softmax function,

$$p(y = i \mid x) = \frac{\exp(z_i)}{\sum_{j=1}^c \exp(z_j)}. \quad (2.2)$$

The network is trained by minimising the cross entropy loss

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{N} \sum_{n=1}^N \log p(y_n \mid x_n; \theta) \quad (2.3)$$

over a training set $\{(x_n, y_n)\}_{n=1}^N$ with mini batch stochastic gradient descent or one of its adaptive variants. In this project, the Adam optimiser [16] is used. Adam keeps an estimate of the first and the second moment of the gradient for every parameter and adapts the effective step size accordingly, which makes the training robust against a suboptimal choice of the learning rate.

An important property of such networks for this project is that the knowledge of the network

is stored in a distributed way. Every weight takes part in the computation of every output, and every output depends on many weights. This property is one of the main reasons for the forgetting problem that is described in Section 2.3.

2.2. Continual Learning

2.2.1. Problem Definition

In continual learning, the training data is not available as one static data set but arrives as a sequence of T tasks $\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_T$. Each task $\mathcal{T}_t$ consists of a data set $\mathcal{D}_t = \{(x_i^{(t)}, y_i^{(t)})\}_{i=1}^{N_t}$ from a task specific distribution. At time t , the learner has only access to $\mathcal{D}_t$, or at most to a very limited memory of earlier tasks, and has to update its parameters from θ_{t-1} to θ_t . The goal is to perform well on all tasks seen so far, not only on the current one.

This setting is fundamentally different from multi task learning, where the data of all tasks is available at the same time and can be mixed freely, and also from transfer learning, where only the performance on a new target task matters and the performance on the source task is allowed to degrade.

2.2.2. The Three Continual Learning Scenarios

Van de Ven and Tolias [14] distinguish three scenarios of increasing difficulty, which differ in the information that is available at test time:

Task incremental learning.

The task identity is known at test time. The model may use task specific components, for example a separate output head per task, and it only has to solve the sub problem of the current task. SplitMNIST with one output head per task belongs to this scenario.

Domain incremental learning.

The task identity is not needed at test time, but all tasks share the same output space. The model must solve every task without knowing which task an input belongs to, but it never has to distinguish classes of different tasks. PermutedMNIST with one shared ten class head is the standard example.

Class incremental learning.

The model additionally has to infer the task identity itself, that is, it has to distinguish all classes of all tasks at the same time. This is the hardest of the three scenarios.

The two benchmarks of this project cover the first two scenarios. This is a deliberate choice, as explained in Section 1.2: the main goal is to study the stability plasticity behaviour of the Mixture of Experts architecture in a clean setting, not to solve the harder class incremental problem, which usually needs additional memory based techniques.

2.2.3. The Stability Plasticity Dilemma

Every continual learning method has to balance plasticity, the ability to gain new knowledge, against stability, the ability to keep old knowledge [12]. Regularisation based methods implement this trade off explicitly with an additional stability term in the loss function. Architectural methods implement it implicitly by assigning different parts of the network to different tasks. The Mixture of Experts approach investigated here realises the trade off through routing: the plasticity is concentrated in the experts that the router selects for the new task, while the remaining experts act as stable memory.

2.3. Catastrophic Forgetting

Catastrophic forgetting denotes the fast and severe loss of previously learned performance when a neural network is trained on new information. The phenomenon was documented systematically by McCloskey and Cohen [1], who showed that networks trained sequentially on two lists of associations perform almost at chance level on the first list after learning the second one. French [2] related the effect to the distributed representations of connectionist models: because all weights take part in the storage of every pattern, writing a new pattern into the network inevitably overwrites parts of the old ones.

From the optimisation point of view, the reason is simple. During the training on task $\mathcal{T}_t$, gradient descent follows the loss landscape of $\mathcal{L}_t$ only. Nothing in the objective prevents the parameters from moving to a region of the weight space that is good for $\mathcal{T}_t$ but arbitrarily bad for the earlier tasks. Because the loss surfaces of different tasks usually do not share a common minimum in a high dimensional weight space, sequential optimisation almost guarantees interference [17].

It is important to note that catastrophic forgetting is not caused by a lack of capacity. Even networks with far more parameters than needed for each single task forget, because the same parameters are reused for all tasks. This observation is the key motivation for the present project: a Mixture of Experts model can partition its capacity so that different tasks rely on different parameters, and can therefore attack the root cause of the interference instead of only dampening its symptoms.

2.4. Metrics for Continual Learning

To make forgetting measurable, the evaluation follows the standard protocol of Lopez-Paz and Ranzato [7]. Let

$$R_{i,j} \in [0, 1] \quad (2.4)$$

be the test accuracy on task $\mathcal{T}_j$ after the model has been trained on task $\mathcal{T}_i$, with $j \leq i$. The triangular matrix R describes the complete learning dynamic of a run. Three scalar metrics are derived from it.

The average accuracy is the mean performance over all tasks after the last training stage,

$$A = \frac{1}{T} \sum_{j=1}^T R_{T,j}. \quad (2.5)$$

It reflects plasticity and stability together and is the primary metric of a continual learner.

The average forgetting measures, for every task, the difference between the best accuracy ever reached on it and its accuracy at the end of training:

$$F = \frac{1}{T-1} \sum_{j=1}^{T-1} \left(\max_{i \in \{j, \dots, T-1\}} R_{i,j} - R_{T,j} \right). \quad (2.6)$$

A value of $F = 0$ means perfect retention; larger positive values mean more severe forgetting.

The backward transfer (BWT) measures the average influence that learning a new task has on the performance on previous tasks,

$$\text{BWT} = \frac{1}{T-1} \sum_{j=1}^{T-1} (R_{T,j} - R_{j,j}). \quad (2.7)$$

Negative values indicate forgetting; positive values would mean that learning later tasks even improved the earlier ones.

2.5. Mixture of Experts

2.5.1. Historical Roots

The Mixture of Experts idea goes back to Jacobs, Jordan, Nowlan and Hinton [9], who proposed to replace one single network by a set of expert networks together with a gating network. Each expert specialises on a region of the input space, and the gating network

learns to assign every input to the most suitable expert. Because the experts are trained jointly but compete through the gate, the architecture shows a form of divide and conquer: a complex problem is decomposed into simpler sub problems that are solved by specialists. Hierarchical variants of the model were analysed afterwards by Jordan and Jacobs [18].

2.5.2. Sparsely Gated Mixture of Experts

The modern, scalable form of the idea is the sparsely gated MoE layer of Shazeer et al. [10]. Instead of weighting all experts, the gate activates only the k best ones, typically $k \in \{1, 2\}$, which keeps the computational cost per input almost constant while the total number of parameters grows with the number of experts. This architecture later became a core component of very large language models, most notably the Switch Transformer [11], and showed that expert based capacity scaling works reliably in practice.

Formally, an MoE layer consists of E expert networks $f_1, \dots, f_E$ and a gating network g . For an input h , the gate first computes logits and converts them into a probability distribution over the experts:

$$g(h) = \text{softmax}(W_g h + b_g) \in \mathbb{R}^E. \quad (2.8)$$

Only the k experts with the largest gate values are evaluated. If $\mathcal{S}(h) = \text{top-}k(g(h))$ denotes the index set of the selected experts and $\tilde{g}_e$ their renormalised gate values, the output of the layer is the convex combination

$$\text{MoE}(h) = \sum_{e \in \mathcal{S}(h)} \tilde{g}_e(h) f_e(h), \quad \tilde{g}_e(h) = \frac{g_e(h)}{\sum_{e' \in \mathcal{S}(h)} g_{e'}(h)}. \quad (2.9)$$

All parameters, those of the experts and those of the gate, are trained jointly by backpropagation; the gate receives gradients through the weights $\tilde{g}_e$.

2.5.3. Load Balancing

A well known failure mode of MoE training is expert collapse: the gate quickly prefers one or two experts, these experts receive more gradient signal and become even better, and the remaining experts starve. To prevent this, an auxiliary loss is used that rewards a uniform usage of the experts [10]. The variant used in this project follows the Switch Transformer [11]. Let f_e be the fraction of samples of a batch whose highest ranked expert is e , and let P_e be the mean gate probability of expert e over the batch. Then

$$\mathcal{L}_{\text{LB}} = E \cdot \sum_{e=1}^E f_e P_e \quad (2.10)$$

is minimised when all experts are used equally. It is added to the task loss with a small coefficient α :

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \mathcal{L}_{\text{LB}}. \quad (2.11)$$

As Chapter 7 will show, this auxiliary loss has an unexpected and important side effect in the continual setting.

2.5.4. Position of the MoE Layer in the Network

A design question that is rarely discussed in the MoE literature, but is central for this project, is where in the network the MoE layer should be placed. In large language models, the MoE layers replace the feed forward blocks in the middle of a deep stack, so the router always sees a hidden representation that was produced by shared layers in front of it. Two extreme positions can be distinguished.

With *hidden level routing*, the input first passes one or more shared layers, and the router assigns the resulting hidden representation to the experts. The shared layers form a common feature extractor for all inputs, and the experts only specialise on the later processing stages. With *input level routing*, the router and the experts act directly on the raw input, and only the mixed expert output passes shared layers.

The difference matters for continual learning, because routing can only protect the parameters that lie behind the router. Parameters in front of the router process every input of every task and are updated by every task, so they behave exactly like the weights of a dense network. If two tasks differ in the way the raw input must be interpreted, as for example two different pixel permutations do, the decisive task specific knowledge lives precisely in this early mapping. A router behind a shared trunk cannot protect this mapping, no matter how well it separates the tasks in the hidden space. This consideration is taken up again in the design decisions of Chapter 4 and becomes directly relevant in the evaluation in Chapter 7.

2.5.5. Why MoE Is Attractive for Continual Learning

The connection between MoE and continual learning is the central idea of this project. Three properties make the architecture promising. First, conditional computation: only a subset of the network is active for any given input, so if the router separates tasks, the gradient updates on a new task touch only a subset of the parameters, which directly limits interference. Second, spare capacity: the total number of parameters scales with the number of experts, not with the cost per input, so new tasks can occupy capacity that was not needed by earlier tasks. Third, learned task inference: the gate itself is a small classifier

that learns, without being told the task identity, to distinguish the input distributions of the tasks. Whether these properties actually appear in a sequential training regime, and how strongly they protect against forgetting, is exactly what the experiments in Chapter 7 are designed to measure.

3. Problem Analysis

The analysis of the problem to be solved is the basis of every engineering like procedure. In this chapter, the overall problem is decomposed into sub problems on the basis of the fundamentals of Chapter 2, the existing solution approaches from the literature are analysed, and the requirements for the system to be developed are derived.

3.1. Decomposition of the Problem

The overall problem, learning a sequence of tasks without losing old knowledge, can be decomposed into the following sub problems:

- SP1. Representation of knowledge.** In which part of the network is the knowledge about a task stored, and how can new knowledge be added without changing the old one?
- SP2. Assignment of tasks to parameters.** Which parameters should be updated when a new task arrives, and who makes this decision?
- SP3. Measurement of forgetting.** How can forgetting be measured objectively, so that different solutions can be compared fairly?
- SP4. Fair comparison.** How can it be guaranteed that an observed improvement comes from the investigated mechanism and not, for example, simply from a larger number of parameters?

These sub problems depend on each other. The assignment of tasks to parameters (SP2) decides directly about the representation of knowledge (SP1): if two tasks share all parameters, interference is unavoidable, as explained in Section 2.3. The measurement (SP3) is needed before any solution can be judged, and the fairness question (SP4) decides which baseline models the experiment needs. The design in Chapter 4 answers SP1 and SP2 with the Mixture of Experts architecture and its router, while the experiment protocol in Chapters 6 and 7 answers SP3 and SP4.

3.2. Analysis of Existing Solution Approaches

Continual learning has been studied intensively in the last years, and several surveys give a systematic overview [3, 4, 19]. The existing methods can be divided into three families.

3.2.1. Regularisation Based Methods

Regularisation based methods leave the network architecture unchanged and modify the loss function instead. The most influential representative is Elastic Weight Consolidation (EWC) [5]. After finishing a task, EWC estimates the importance of every parameter by the diagonal of the Fisher information matrix,

$$F_{ii} = \mathbb{E} \left[\left(\frac{\partial \log p(y | x; \theta)}{\partial \theta_i} \right)^2 \right], \quad (3.1)$$

and afterwards punishes quadratic deviations of important parameters from their stored values θ_i^* :

$$\mathcal{L}_{\text{EWC}}(\theta) = \mathcal{L}_t(\theta) + \frac{\lambda}{2} \sum_i F_{ii} (\theta_i - \theta_i^*)^2. \quad (3.2)$$

The coefficient λ controls the stability plasticity trade off. Related methods estimate the importance differently: Synaptic Intelligence accumulates an online importance measure along the optimisation path [20], and Memory Aware Synapses estimates the importance from the sensitivity of the output function [21]. A shared limitation is that the quadratic penalty is only a local approximation: with more and more tasks, the collected constraints can rigidify the network and finally prevent it from learning new tasks at all. A second branch of regularisation avoids storing old data by distillation, for example Learning without Forgetting [22].

3.2.2. Replay Based Methods

Replay methods keep a small episodic memory of real or generated samples from earlier tasks and mix them into the training of new tasks. iCaRL [6] combines an exemplar memory with a nearest mean classifier. Gradient Episodic Memory [7] uses the memory to constrain the gradient updates so that the loss on every remembered task cannot increase. Generative replay replaces the stored samples with data from a learned generative model [23]. Replay belongs to the strongest strategies, but it needs to store data, which can be impossible for privacy reasons, and the memory grows with the number of tasks.

3.2.3. Architecture Based Methods

The third family changes the network itself. Progressive Neural Networks [8] allocate a new network column per task and connect it to the frozen columns of earlier tasks; forgetting is impossible by construction, but the number of parameters grows linearly with the number of tasks, and the task identity must be known at test time. Other approaches assign a

binary mask to every task and train only the unmasked weights [24]. Expert Gate [25] is conceptually closest to this project: it also uses a network of experts with a gating mechanism, but it trains one new expert per task and selects experts by the task identity, while the approach here lets the router learn the assignment fully automatically without any task labels.

3.2.4. Comparison of the Method Families

Table 3.1 contrasts the three families along the criteria that matter for this project. No family dominates the others. Regularisation methods are cheap and need neither stored data nor task labels, but their protection is only a local approximation of the true loss landscape, and the accumulated constraints of many tasks can finally block further learning. Replay methods reach the strongest results in most benchmarks, but they store old data or a generative model of old data, which can be impossible under privacy constraints, and their memory requirement grows with the length of the task sequence. Architectural methods avoid both problems, but the existing representatives either grow the network with every task or need the task identity at test time to select the right component.

Table 3.1.: Comparison of the three continual learning method families.

Criterion	Regularisation	Replay	Architecture
Stores old data	no	yes	no
Needs task identity	no	no	often
Capacity growth per task	none	memory only	often linear
Protection mechanism	approximate	strong	structural
Representative methods	EWC, SI, MAS	iCaRL, GEM	PNN, Expert Gate

For the present project, the decisive criterion is the combination of the first three rows: a method that stores no data, needs no task labels and does not grow without limit. Only a softly partitioned architecture can offer all three properties at the same time. A Mixture of Experts model with a learned router is a natural candidate, because its capacity is fixed in advance but is used conditionally, so new tasks can occupy experts that earlier tasks rarely touch.

3.2.5. Conclusion for this Project

The analysis shows a gap: regularisation methods are efficient but only approximate, replay methods are strong but need stored data, and existing architectural methods either grow without limit or need the task identity. A Mixture of Experts model with a learned router

promises a middle way: no stored data, no task labels, and a capacity that is partitioned softly instead of being fixed by hand. This promise is tested in the present project.

3.3. Requirements Analysis

From the problem analysis, the following requirements for the system were collected and prioritised in four categories, as it is common practice for project work.

Must have requirements.

- **M1:** Sequential training of a model on a sequence of tasks without access to the data of earlier tasks.
- **M2:** At least two continual learning benchmarks built from MNIST (SplitMNIST and PermutedMNIST).
- **M3:** A working Mixture of Experts model with a trainable router and more than two experts.
- **M4:** A naive baseline with the same training protocol for comparison.
- **M5:** Quantitative evaluation with the standard metrics average accuracy, average forgetting and backward transfer after every training stage.

Should have requirements.

- **S1:** A regularisation baseline (EWC) to position the MoE results relative to an established method.
- **S2:** A capacity matched wide baseline to separate capacity effects from routing effects (answers SP4).
- **S3:** Reproducibility through fixed random seeds and repeated runs.
- **S4:** Analysis of the router (expert usage per task).

Could have requirements.

- **C1:** An auxiliary loss for load balancing that keeps all experts active.
- **C2:** Configurable hyperparameters (number of experts, top k , epochs, learning rate).

Nice to have requirements.

- **N1:** Extension to further benchmarks or to the class incremental scenario.
- **N2:** Dynamic addition of new experts per task.

All must have and should have requirements were realised, and C1 and C2 were realised as well. N1 and N2 remain future work and are discussed in Section 8.2.

4. Solution Concept

On the basis of the problem analysis of Chapter 3 and the theoretical knowledge of Chapter 2, this chapter develops the solution concept. Section 4.1 gives an overview of the system architecture. Section 4.2 specifies the Mixture of Experts model, Section 4.3 the router, and Section 4.4 the continual training procedure.

4.1. Architectural Overview

The system is organised in four loosely coupled layers, which are shown in Figure 4.1 together with their data flow.

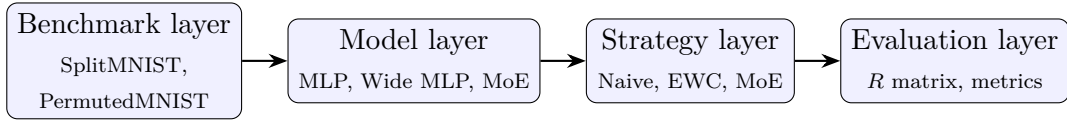


Figure 4.1.: Layered concept of the experimental system. The data flows from the benchmark layer through the model and strategy layers to the evaluation layer, which produces the accuracy matrix, the scalar metrics and the router profiles.

The benchmark layer turns a static data set into a sequence of tasks and provides one training and one test data loader per task. The model layer contains the network architectures: the baseline MLP, the wide MLP and the Mixture of Experts network. The strategy layer implements the training procedures, namely naive fine tuning, EWC and the MoE training with the auxiliary loss. Finally, the evaluation layer computes the accuracy matrix R after every training stage, derives the scalar metrics from it and extracts the router profiles for the MoE model.

This separation had two practical advantages during the project. First, every component could be implemented and tested independently. Second, the comparison between the approaches stays fair, because all approaches consume exactly the same task sequence and are evaluated by exactly the same code. This directly answers the fairness sub problem SP4 of Section 3.1.

4.2. Concept of the MoE Model

The proposed model follows the structure sketched in Figure 4.2. A shared trunk maps the flattened 28×28 input image to a 400 dimensional representation. This representation is

processed by one MoE layer consisting of $E = 8$ experts and a router. Each expert is a small feed forward block with one hidden layer and ReLU activation, with identical input and output dimension, so that the expert outputs can be combined by a weighted sum. The router selects the $k = 2$ experts with the highest gate values for every sample and combines their outputs according to Equation (2.9). On top of the MoE layer, one linear output head per task produces the class logits (multi head protocol, see Section 2.2.2).

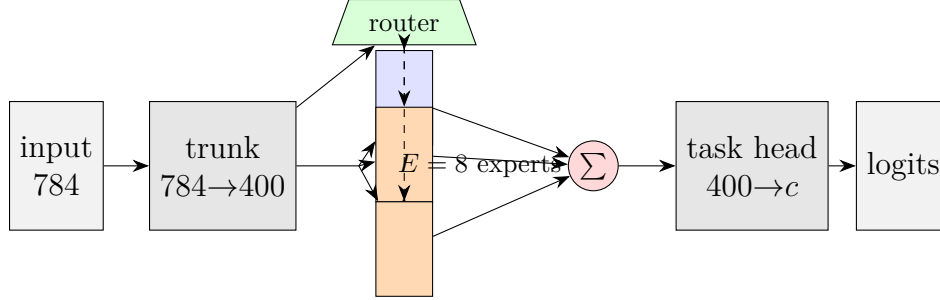


Figure 4.2.: Concept of the MoE model. A shared trunk feeds eight parallel experts. The router (dashed arrows) selects the top 2 experts per sample (highlighted); their outputs are combined as a gate weighted sum and passed to the output head of the current task.

The choice $E = 8$ with $k = 2$ follows the efficiency requirement of Section 1.2: with five tasks, eight experts give enough spare capacity for the router to separate the tasks, while the activated computation per sample, namely two experts, stays close to that of the baseline. The output head per task keeps the comparison with the baselines protocol fair, because the baselines use identical heads.

4.3. Concept of the Router

The router is a single linear layer $W_g \in \mathbb{R}^{8 \times 400}$ followed by a softmax, see Equation (2.8). It is deliberately kept simple: a small gate cannot itself become a bottleneck of forgetting, and its decisions stay easy to interpret and to visualise. Two design decisions are essential for the continual setting:

- D1.** The router does not receive the task identity. The gate only sees the hidden representation of the input. Every task separation it shows is therefore learned from the data alone, which makes the router analysis in Chapter 7 meaningful.
- D2.** The router is trained only on the current task, without any mechanism that forces it to keep old routing decisions. If the gate changed its assignments freely, previously acquired expert knowledge would be re-routed and lost. The experiments test whether the assignments stay stable enough in practice.

The auxiliary loss for load balancing from Equation (2.10), with the coefficient $\alpha = 0.01$, prevents expert collapse at the beginning of training and guarantees that all eight experts stay functional, so that later tasks can actually find unused capacity. As the evaluation will show, this component behaves differently than expected in the continual setting, which leads to the architecture variant described in Section 7.5.

Considered alternatives. Two alternative router designs were considered and rejected. The first alternative was a nonlinear gate with one hidden layer. It was rejected because the gate itself is part of the continual learning problem: a larger gate has more parameters that can forget, and its decisions are harder to interpret in the router analysis. The second alternative was a hard top one routing without any mixing, as in the Switch Transformer [11]. It was rejected for this small scale setting because the gradient signal of a single expert per sample is noisy, and because a soft mixing of two experts makes the transition between tasks smoother when the router changes its assignment. The top two choice keeps most of the sparsity but keeps training stable, which was confirmed in preliminary tests.

Expected behaviour. If the concept works as intended, the following behaviour should be observable in the experiments. During the first task, the router distributes the samples over several experts. When the second task arrives, the router should prefer experts that carry little knowledge of the first task, so that the gradient updates of the new task mainly rewrite unused or weakly used parameters. Over the whole sequence, a pattern of task specific expert subsets should emerge, and the pairwise routing similarity between tasks should stay low. These predictions are checked explicitly in Section 7.6.

4.4. Concept of the Training Procedure

All approaches follow the same sequential protocol, which is shown in Table 4.1. For every task, the model is trained from its current state on the training data of that task; afterwards, it is evaluated on the test data of all tasks seen so far, which produces one row of the accuracy matrix R . No data of earlier tasks is shown again, and no task boundaries are signalled to the router.

Table 4.1.: Sequential continual training protocol used for all approaches.

Algorithm: Sequential training over a task sequence

Input: task sequence $(\mathcal{T}_1, \dots, \mathcal{T}_T)$; model f_θ ; strategy specific loss $\mathcal{L}$ 1: **for** $t = 1$ **to** T **do**2: train f_θ on $\mathcal{T}_t$ by minimising $\mathcal{L}$ 3: **for** $j = 1$ **to** t **do**4: $R_{t,j} \leftarrow$ accuracy of f_θ on the test set of $\mathcal{T}_j$ 5: **end for**6: **end for**7: derive A , F and BWT from R (Equations (2.5) to (2.7))

For the EWC strategy, step 2 additionally estimates the diagonal Fisher information on the current task after convergence and accumulates the quadratic penalty of Equation (3.2) for all later tasks. For the MoE strategy, step 2 minimises the combined loss of Equation (2.11). The evaluation in lines 3 to 5 is identical for all approaches, which guarantees that differences in the metrics can be attributed to the models and strategies alone.

5. Implementation

This chapter describes the concrete implementation of the solution concept from Chapter 4. Section 5.1 lists the used development tools, Section 5.2 describes the module structure of the code, and Sections 5.3 to 5.5 explain the implementation of the benchmarks, the models and the training strategies. Only one short code excerpt is shown; the complete source code is printed in Appendix A.

5.1. Development Tools

The system was implemented in Python 3.12. As deep learning framework, PyTorch 2.8 [26] with torchvision for data loading was used. NumPy was used for numerical post processing and Matplotlib for the visualisation of the results. All experiments ran on the CPU of a standard Linux machine; no GPU was necessary, because the models and data sets are small. Every module is documented by docstrings and inline comments.

5.2. Module Structure

The code is organised as a small package with one module per layer of the concept in Figure 4.1:

- `datasets.py`: construction of the SplitMNIST and PermutedMNIST task sequences (benchmark layer);
- `models.py`: the baseline MLP, the wide MLP, the MoE network and the input level MoE variant (model layer);
- `continual.py`: the training strategies naive, EWC and MoE, together with the evaluation metrics (strategy and evaluation layer);
- `run_experiments.py`, `run_ablation.py` and `run_v2.py`: the experiment drivers for the main experiments, the load balancing ablation and the architecture variant;
- `make_figures.py` and `make_tables.py`: generation of all figures and result tables of this report directly from the stored results.

5.3. Benchmark Construction

5.3.1. SplitMNIST

The ten MNIST classes are split into five consecutive tasks $\{0, 1\}, \{2, 3\}, \dots, \{8, 9\}$. For every task, the samples of the two classes are extracted from the original training and test sets, the labels are remapped to the local classes $\{0, 1\}$ of the task specific output head, and the images are flattened to vectors of 784 float values. The result is a task incremental benchmark in the multi head protocol of Section 2.2.2.

5.3.2. PermutedMNIST

For PermutedMNIST, the first task is the original MNIST data set. For every later task, a fixed random permutation of the 784 pixel positions is drawn from a seeded random number generator and applied to all images of that task. Because the classes stay the same in every task, all tasks share one ten class output head, which realises the domain incremental scenario. The permutation is applied on the fly when a sample is loaded, so the base tensors are stored only once and the memory usage stays low even for many tasks.

5.4. Model Implementations

The baseline network is a two hidden layer MLP with $784 \rightarrow 400 \rightarrow 400$ units and ReLU activations, plus one linear output head per task. The wide baseline increases the hidden dimension to 1100, which raises its parameter count slightly above the MoE model and thereby implements the capacity control required in Section 3.3. Table 5.1 lists the resulting parameter counts.

Table 5.1.: Trainable parameters of the compared architectures (with five output heads on SplitMNIST).

Architecture	Parameters
Baseline MLP (784→400→400)	476,004
MoE, hidden level routing (8 experts, top 2)	1,602,012
MoE, input level routing (8 experts, top 2)	1,679,752
Wide MLP (784→1100→1100)	2,074,600

The core of the project is the MoE layer. Listing 5.1 shows its forward pass. The router computes a softmax distribution over the eight experts, the two largest gate values are renormalised, and only the selected experts are evaluated. The outputs are added with

their gate values as weights. The input level variant, which resulted from the evaluation in Chapter 7, uses the same gating mechanics but places the router and the experts directly at the raw input instead of behind the shared trunk.

```

1 def forward(self, x):
2     # compute the gate distribution over all experts
3     logits = self.router(x)
4     gates = F.softmax(logits, dim=-1)
5     topv, topi = gates.topk(self.top_k, dim=-1)
6     topv = topv / topv.sum(dim=-1, keepdim=True)
7
8     # evaluate only the selected experts and mix their outputs
9     out = torch.zeros_like(x)
10    for slot in range(self.top_k):
11        idx = topi[:, slot]
12        w = topv[:, slot].unsqueeze(1)
13        for e in range(self.n_experts):
14            mask = idx == e
15            if mask.any():
16                out[mask] += w[mask] * self.experts[e](x[mask])
17    return out, gates

```

Listing 5.1: Forward pass of the sparsely gated MoE layer (excerpt from `models.py`).

5.5. Training Strategies

5.5.1. Naive Fine Tuning

The naive trainer minimises the cross entropy loss of Equation (2.3) with the Adam optimiser, learning rate 10^{-3} , for five epochs per task. It deliberately does nothing to protect old knowledge and therefore measures the amount of forgetting that every continual learning mechanism has to beat.

5.5.2. Elastic Weight Consolidation

The EWC trainer extends the naive trainer in two places. After finishing a task, it estimates the diagonal Fisher information of Equation (3.1) from the gradients on the training data of that task and stores a copy of the current parameters. During every later task, the quadratic penalty of Equation (3.2) with $\lambda = 500$ is added to the loss. The penalty accumulates over tasks, so that all previous tasks are protected at the same time. Care had to be taken with the lazily created output heads: heads of other tasks do not take part in the current computation graph, so their Fisher contribution is skipped.

5.5.3. MoE Training

The MoE trainer minimises the combined loss of Equation (2.11) with the same optimiser settings as the other strategies. After the complete task sequence has been learned, it extracts the router profile: the mean gate probability of every expert, computed over the test set of every task. This 5×8 matrix is the raw material for the router analysis in Section 7.6.

5.6. Experiment Drivers and Result Storage

Three driver scripts connect the modules to complete experiments. The main driver `run_experiments.py` trains all four approaches on both benchmarks with three seeds each. Before every run, the random seed is set explicitly, so that the initialisation of every model and the data order are identical across approaches and repeatable across machines. After every single run, the accuracy matrix, the derived metrics, the parameter count and the runtime are appended to the result file, and the router profile is stored as a separate NumPy archive. The results are written to disk after every run instead of only at the end, so a crash or an out of memory stop cannot destroy the measurements that were already collected. This design decision proved its value during commissioning, as Section 6.2 describes.

The second driver `run_ablation.py` repeats the MoE training on PermutedMNIST with smaller load balancing coefficients, and the third driver `run_v2.py` evaluates the input level architecture variant on both benchmarks. All three drivers share the same trainer and metric code, so the results are directly comparable.

A deliberate property of the implementation is that no number in this report was copied by hand. The two scripts `make_figures.py` and `make_tables.py` read the stored result files and generate every figure of Chapter 7 as a vector graphic and every result table as a LaTeX file with the measured values. This closes the gap between the experiment and the document and removes a common source of transcription errors.

6. Commissioning

The task of this chapter is to show how the implemented solution was brought into operation. For this project, this means the preparation of the runtime environment, the execution of the experiment suite, and the steps taken to make the results reproducible.

6.1. Preparation of the Environment

The experiments were carried out on a standard Linux machine with a CPU only installation. The environment consists of Python 3.12 with the packages PyTorch 2.8, torchvision, NumPy and Matplotlib. The MNIST raw files were downloaded once from the official mirror and stored in the project folder, so that the experiments can run without further downloads. The installation and execution requires only the following commands:

```
1 pip install torch torchvision numpy matplotlib
2 cd moe_continual_learning/src
3 python3 run_experiments.py      # main experiment suite
4 python3 run_ablation.py        # load balancing ablation
5 python3 run_v2.py              # input level architecture variant
6 python3 make_figures.py        # figures for the report
7 python3 make_tables.py         # LaTeX tables for the report
```

Listing 6.1: Installation and execution of the experiment suite.

6.2. Execution of the Experiments

The main experiment driver runs the full comparison matrix: four approaches (naive MLP, MLP with EWC, wide MLP, MoE), two benchmarks (SplitMNIST and PermutedMNIST) and three random seeds, which results in 24 complete continual training runs. Before the construction of every model, the global random seed is reset, so that all approaches start from comparable initial conditions; the task permutations of PermutedMNIST are also derived from the seed. After every training stage, the current accuracy row is printed and stored, so the progress can be followed in the log file. A typical log line looks as follows:

```
1 [split_mnist] seed=0 MoE: avg_acc=0.9523 forgetting=0.0561 bwt=-0.0561
2 [permuted_mnist] seed=2 MLP + EWC: avg_acc=0.9490 forgetting=0.0354
```

Listing 6.2: Example lines of the experiment log.

The results are written to disk after every single run. This proved to be important in practice: during the project, one experiment process was terminated by the operating system because of high memory usage, and thanks to the incremental storage, no completed run had to be repeated. After this incident, the PermutedMNIST loader was changed to apply the permutation on the fly instead of materialising a copy of the whole data set per task, which reduced the memory consumption of the benchmark clearly.

The runtime of a single continual training run over five tasks lies between 18 s (naive MLP on SplitMNIST) and 430 s (MoE on PermutedMNIST) on the CPU. The complete main suite takes about one hour, the ablation about 20 minutes and the architecture variant about 25 minutes.

6.3. Reproducibility Measures

Several measures were taken so that every number in this report can be traced back to a concrete training run. All random choices are controlled by seeds: the initialisation of every model, the data order of every training run and the pixel permutations of PermutedMNIST. As a consequence, repeated executions of the suite produce identical results, which was verified explicitly when the suite had to be restarted after the incident described above. The raw results of every run are stored as JSON files, and the router profiles are stored separately as compressed NumPy archives. Finally, all figures and all result tables of Chapter 7 were generated directly from these stored results by the two scripts `make_figures.py` and `make_tables.py`; no number was copied by hand.

7. Evaluation

The task of the evaluation is to check to what extent the objectives of the work were reached. Section 7.1 describes the experimental setup, Sections 7.2 and 7.3 report the results on SplitMNIST and PermutedMNIST, Section 7.4 analyses why the first MoE architecture fails on PermutedMNIST, Section 7.5 evaluates the resulting architecture variant, Section 7.6 analyses the behaviour of the router, and Section 7.7 discusses the findings.

7.1. Experimental Setup

Both benchmarks consist of five tasks and were constructed exactly as described in Section 5.3. SplitMNIST is evaluated in the task incremental multi head protocol, PermutedMNIST in the domain incremental protocol with one shared ten class head. All approaches see the identical task sequence, the identical data order and the identical pixel permutations, because all random choices are derived from controlled seeds.

Four approaches are compared in the main experiment: the naive MLP, the MLP with EWC ($\lambda = 500$), the wide naive MLP and the MoE model with hidden level routing (eight experts, top 2, load balancing coefficient $\alpha = 0.01$). Every model is trained with the Adam optimiser, learning rate 10^{-3} , for five epochs per task with a mini batch size of 256. All experiments are repeated with the three seeds $\{0, 1, 2\}$, and all reported numbers are means over these seeds with the standard deviation.

After every training stage, the model is evaluated on the test sets of all tasks seen so far, which gives the accuracy matrix R . From the final matrix, the average accuracy A , the average forgetting F and the backward transfer BWT are derived, as defined in Section 2.4. Figure 7.1 summarises the two most important metrics for all approaches and both benchmarks.

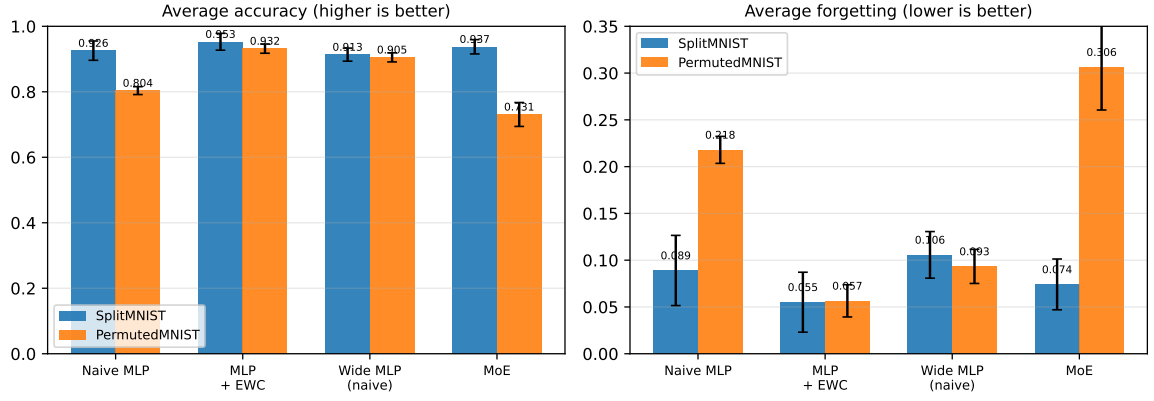


Figure 7.1.: Average accuracy (left) and average forgetting (right) of all compared approaches on both benchmarks (mean over three seeds, error bars show the standard deviation).

The computational cost of the approaches differs noticeably, which is part of the practical trade off. Table 7.1. lists the mean runtime of one complete continual run (five tasks, five epochs per task) on the CPU. The MoE model costs about three times as much as the naive baseline on SplitMNIST and about six times as much on PermutedMNIST, because the expert loop of the current implementation evaluates the selected experts in separate small matrix operations. In a vectorised implementation, as it is used in large scale MoE models, this overhead would be much smaller, because the cost per sample only depends on the two active experts and not on the total number of experts. EWC costs only slightly more than the naive baseline; its price is the additional memory for the Fisher information and the stored parameters, which doubles the state per parameter.

Table 7.1.: Mean runtime of one continual run per approach on the CPU (mean over three seeds).

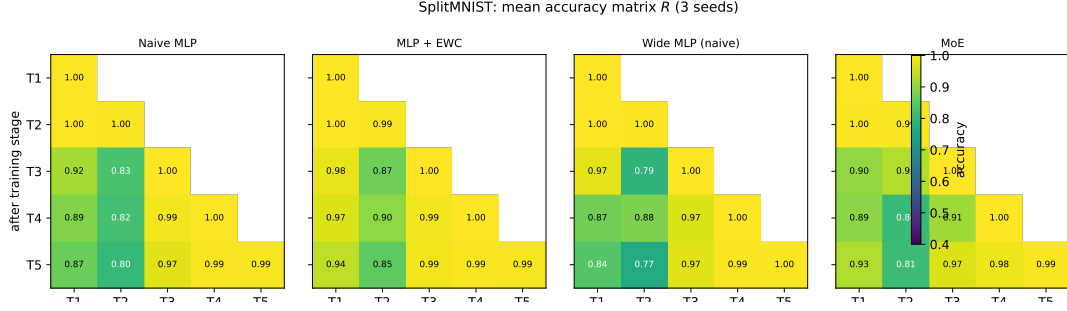
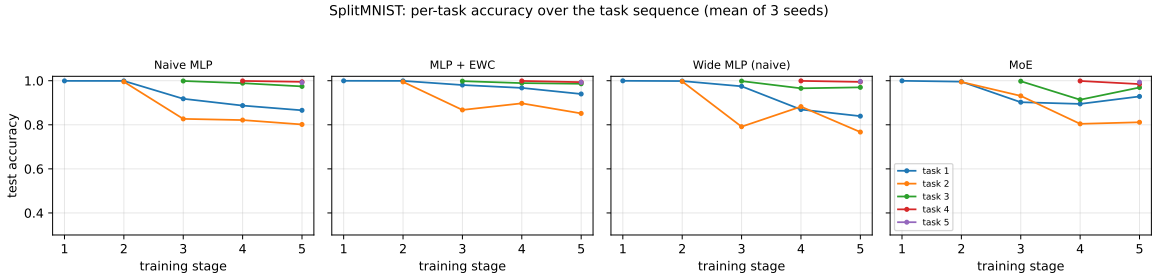
Approach	SplitMNIST	PermutedMNIST
Naive MLP	21 s	101 s
MLP + EWC	26 s	129 s
Wide MLP (naive)	64 s	336 s
MoE	68 s	662 s

7.2. Results on SplitMNIST

Table 7.2. reports the summary metrics on SplitMNIST, Figure 7.2 shows the mean accuracy matrices, and Figure 7.3 shows how the accuracy of every task develops during training.

Table 7.2.: Results on SplitMNIST (mean $\pm$ standard deviation over three seeds; best value per column in bold).

Approach	Avg. accuracy $\uparrow$	Avg. forgetting $\downarrow$	BWT $\uparrow$
Naive MLP	0.9256 ± 0.0295	0.0890 ± 0.0375	-0.0890 ± 0.0375
MLP + EWC	0.9528 ± 0.0257	0.0551 ± 0.0320	-0.0551 ± 0.0320
Wide MLP (naive)	0.9135 ± 0.0199	0.1057 ± 0.0249	-0.1057 ± 0.0249
MoE	0.9375 ± 0.0218	0.0741 ± 0.0271	-0.0741 ± 0.0271

**Figure 7.2.:** Mean accuracy matrix R on SplitMNIST. Entry (i, j) is the test accuracy on task j after training task i ; a column that stays bright towards the bottom indicates good retention of that task.**Figure 7.3.:** Development of the per task test accuracy on SplitMNIST over the five training stages (mean of three seeds).

Three observations stand out. First, forgetting is clearly visible in the unprotected baselines: the naive MLP loses on average 8.9 percentage points of previously acquired accuracy, and the wide MLP loses even more (10.6 points), although both keep the accuracy of the current task close to perfect while they train on it. The forgetting is concentrated in the second task, whose accuracy drops from about 0.99 to about 0.80 in the naive model.

Second, the MoE model reduces the forgetting to 7.4 points and reaches an average accuracy of 0.9375, clearly above the naive baseline (0.9256) and the wide baseline (0.9135). Since the wide baseline has about 30 percent more parameters than the MoE model but performs worst of all, the advantage of the MoE model cannot be explained by its larger capacity. It must come from the routed structure itself, which answers the fairness question SP4 of Section 3.1 for this benchmark.

Third, EWC remains the strongest method on this benchmark (0.9528 average accuracy, 5.5 points forgetting). The MoE model closes roughly half of the gap between the naive baseline and EWC without any stored importance statistics or penalty terms, purely through its architecture.

7.3. Results on PermutedMNIST

Table 7.3. and Figures 7.4 and 7.5 show the corresponding results on PermutedMNIST, the harder of the two benchmarks.

Table 7.3.: Results on PermutedMNIST (mean $\pm$ standard deviation over three seeds; best value per column in bold).

Approach	Avg. accuracy $\uparrow$	Avg. forgetting $\downarrow$	BWT $\uparrow$
Naive MLP	0.8036 $\pm$ 0.0121	0.2179 $\pm$ 0.0144	-0.2179 $\pm$ 0.0144
MLP + EWC	0.9318 $\pm$ 0.0141	0.0566 $\pm$ 0.0172	-0.0566 $\pm$ 0.0172
Wide MLP (naive)	0.9051 $\pm$ 0.0139	0.0933 $\pm$ 0.0182	-0.0933 $\pm$ 0.0182
MoE	0.7307 $\pm$ 0.0365	0.3063 $\pm$ 0.0458	-0.3063 $\pm$ 0.0458

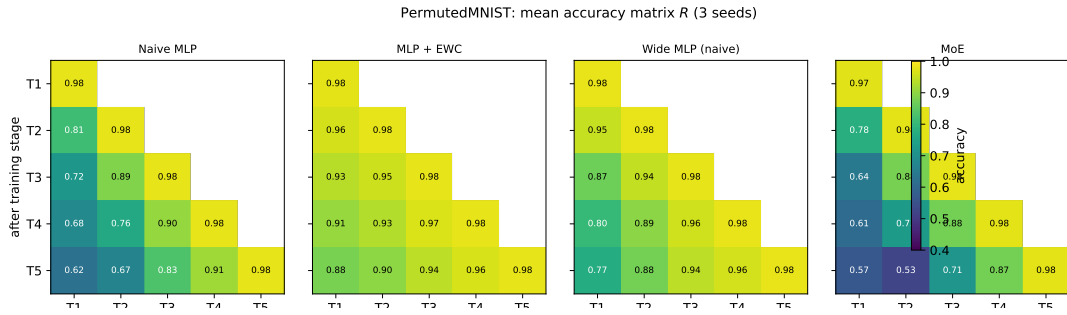


Figure 7.4.: Mean accuracy matrix R on PermutedMNIST.

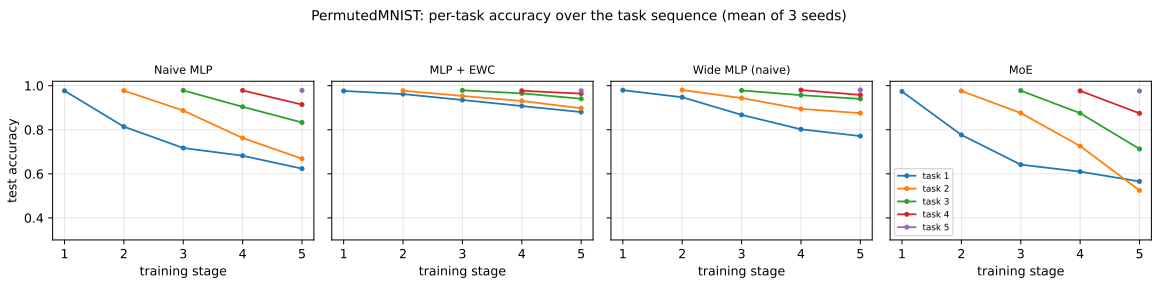


Figure 7.5.: Development of the per task test accuracy on PermutedMNIST over the five training stages (mean of three seeds).

On this benchmark, the naive baseline forgets heavily (21.8 points, average accuracy 0.8036), and EWC again protects effectively (5.7 points, average accuracy 0.9318). Two results, however, were not expected when the experiments were planned:

- F1.** The wide baseline beats the naive baseline clearly (0.9051 versus 0.8036 average accuracy). On a domain incremental benchmark, where every task reuses the same classes, simply having more than four times the parameters already absorbs a large part of the interference. Capacity alone is therefore a strong competitor on PermutedMNIST, much stronger than on SplitMNIST, where the wide baseline was worst.
- F2.** The MoE model with the standard load balancing setting performs worst of all approaches (average accuracy 0.7307, forgetting 30.6 points). This contradicted the project hypothesis and motivated the analysis in the next section.

7.4. Analysis of the PermutedMNIST Failure

The weak PermutedMNIST result of the MoE model was investigated in two steps. The first suspect was the auxiliary loss for load balancing from Equation (2.10). It pushes the router to use all experts uniformly within every batch, and because every task is trained separately, this pressure acts per task: every task is individually encouraged to spread its samples over all eight experts. But if every task uses every expert, no expert is task specific, and the very mechanism that should protect old knowledge is switched off.

To test this explanation, the MoE model was retrained on PermutedMNIST with a ten times weaker coefficient ($\alpha = 0.001$) and completely without the auxiliary loss ($\alpha = 0$), again with three seeds. Table 7.4. shows the results.

Table 7.4.: Ablation of the load-balancing coefficient α on PermutedMNIST (mean $\pm$ std over three seeds).

MoE variant	Avg. accuracy $\uparrow$	Avg. forgetting $\downarrow$	BWT $\uparrow$
MoE ($\alpha = 0.01$, main setting)	0.7307 ± 0.0365	0.3063 ± 0.0458	-0.3063 ± 0.0458
MoE ($\alpha = 0.0$)	0.7935 ± 0.0103	0.2293 ± 0.0127	-0.2293 ± 0.0127
MoE ($\alpha = 0.001$)	0.7493 ± 0.0101	0.2851 ± 0.0129	-0.2851 ± 0.0129

The ablation confirms the load balancing effect, but only partly. Weakening the coefficient improves the metrics slightly, and removing the auxiliary loss completely brings the MoE model back to the level of the naive baseline (average accuracy 0.7935 versus 0.8036). The uniform usage pressure therefore really harms the task specialisation, and the monotonic improvement across the three coefficients makes this dose dependent effect clearly visible. However, even without any load balancing loss, the MoE model does not beat the naive baseline on this benchmark. A second, deeper cause must exist.

A closer look at the architecture reveals it. In the hidden level design of Figure 4.2, the router sits behind a shared trunk. On PermutedMNIST, the difference between two tasks

is exactly a different wiring of the input pixels, that is, the task specific information lives in the input side mapping, which is stored in the shared trunk in front of the router. Every new task must rewrite the weights of the trunk, and the router cannot protect them, because all tasks pass through the same trunk no matter which experts they use afterwards. On SplitMNIST, by contrast, all tasks share the same pixel statistics, so one trunk serves all tasks well and routing behind the trunk is sufficient.

This analysis makes a testable prediction: moving the MoE layer to the input, so that the input side mapping itself is distributed over experts, should protect the part of the network where PermutedMNIST tasks actually differ. The next section tests this prediction.

7.5. Architecture Variant with Input Level Routing

To test the prediction of Section 7.4, a second MoE architecture was implemented. In this variant, the router reads the raw input and distributes eight experts over the 784 dimensional input space, with an expert dimension of 256, followed by a small shared hidden layer and the usual output heads. The parameter count of 1.68 million stays comparable to the hidden level MoE (1.60 million) and below the wide baseline (2.07 million). Input level routing is also plausible from the router’s point of view: the permutations of PermutedMNIST are linearly well separable, so a linear gate on raw pixels can learn to tell the tasks apart, while the same gate behind a shared trunk cannot see the difference that the trunk has already mixed. The training protocol, the hyperparameters and the load balancing setting ($\alpha = 0.01$) are identical to the main experiment; only the position of the MoE layer changes. Table 7.5. compares the variant with all other approaches on both benchmarks.

Table 7.5.: Input-level MoE compared to all approaches on both benchmarks (mean $\pm$ std over three seeds).

Benchmark	Approach	Avg. accuracy $\uparrow$	Avg. forgetting $\downarrow$	BWT $\uparrow$
PermutedMNIST	Naive MLP	0.8036 ± 0.0121	0.2179 ± 0.0144	-0.2179 ± 0.0144
	MLP + EWC	0.9318 ± 0.0141	0.0566 ± 0.0172	-0.0566 ± 0.0172
	Wide MLP (naive)	0.9051 ± 0.0139	0.0933 ± 0.0182	-0.0933 ± 0.0182
	MoE	0.7307 ± 0.0365	0.3063 ± 0.0458	-0.3063 ± 0.0458
	MoE (input-level)	0.7836 ± 0.0313	0.2344 ± 0.0388	-0.2344 ± 0.0388
SplitMNIST	Naive MLP	0.9256 ± 0.0295	0.0890 ± 0.0375	-0.0890 ± 0.0375
	MLP + EWC	0.9528 ± 0.0257	0.0551 ± 0.0320	-0.0551 ± 0.0320
	Wide MLP (naive)	0.9135 ± 0.0199	0.1057 ± 0.0249	-0.1057 ± 0.0249
	MoE	0.9375 ± 0.0218	0.0741 ± 0.0271	-0.0741 ± 0.0271
	MoE (input-level)	0.9647 ± 0.0149	0.0391 ± 0.0183	-0.0391 ± 0.0183

The results confirm the prediction of Section 7.4. On PermutedMNIST, the input level MoE clearly improves over the hidden level variant: the average accuracy rises from 0.731

to 0.784 and the average forgetting drops from 0.306 to 0.234. This recovery of a large part of the deficit supports the diagnosis that the shared trunk in front of the router was the weak point, because routing the raw input protects exactly the input side mapping that PermutedMNIST overwrites. At the same time the variant stays slightly below the naive baseline on this benchmark, so input level routing alone does not fully solve the domain incremental case; the remaining gap suggests that the shared hidden layer behind the experts still mixes information from all permutations. On SplitMNIST, the variant achieves the best result of all compared approaches, with an average forgetting of only 3.9 points, better than EWC. The two variants together show that the position of the router in the architecture decides whether the MoE idea works in a continual setting: routing protects only the parameters behind the router, so the router has to sit where the tasks differ.

7.6. Router Analysis

The router profiles, that is, the mean gate probability of every expert on the test data of every task, make the mechanism behind these results directly visible. Figure 7.6 shows the profile on SplitMNIST for the hidden level model, Figure 7.7 shows the profile on PermutedMNIST for the hidden level model with the main setting, Figure 7.8 shows the profile of the input level variant on PermutedMNIST, and Figure 7.9 quantifies the pairwise similarity of the per task routing distributions by their cosine similarity.

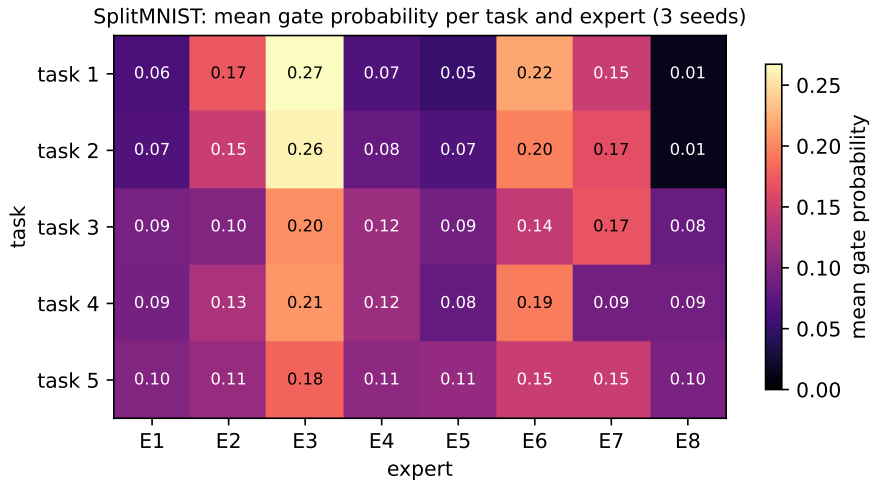


Figure 7.6.: Router profile of the hidden level MoE on SplitMNIST: mean gate probability of every expert for every task (mean over three seeds).

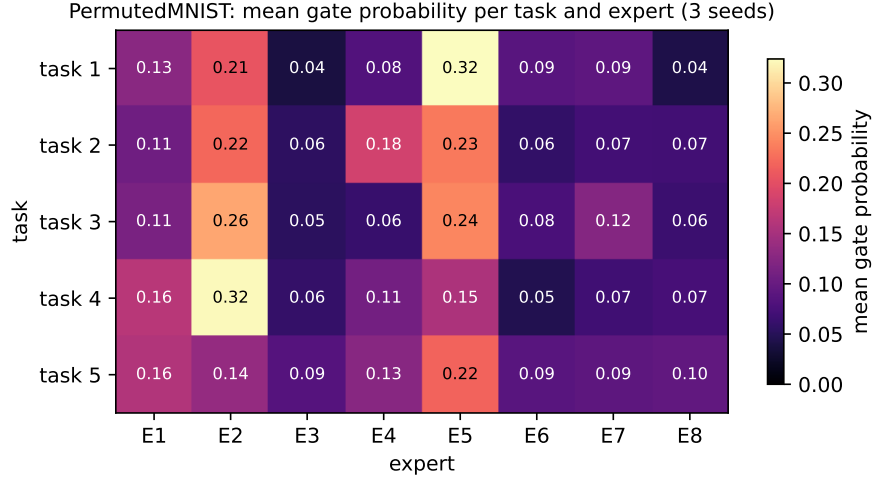


Figure 7.7.: Router profile of the hidden level MoE on PermutedMNIST with the main setting. All tasks concentrate on the same two experts E2 and E5, which explains the strong forgetting on this benchmark.

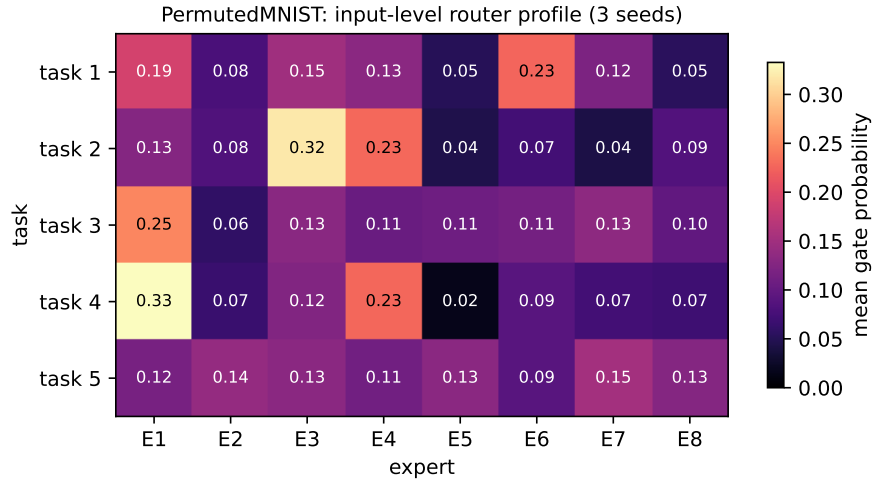


Figure 7.8.: Router profile of the input level MoE on PermutedMNIST. The tasks are distributed over clearly different expert subsets.

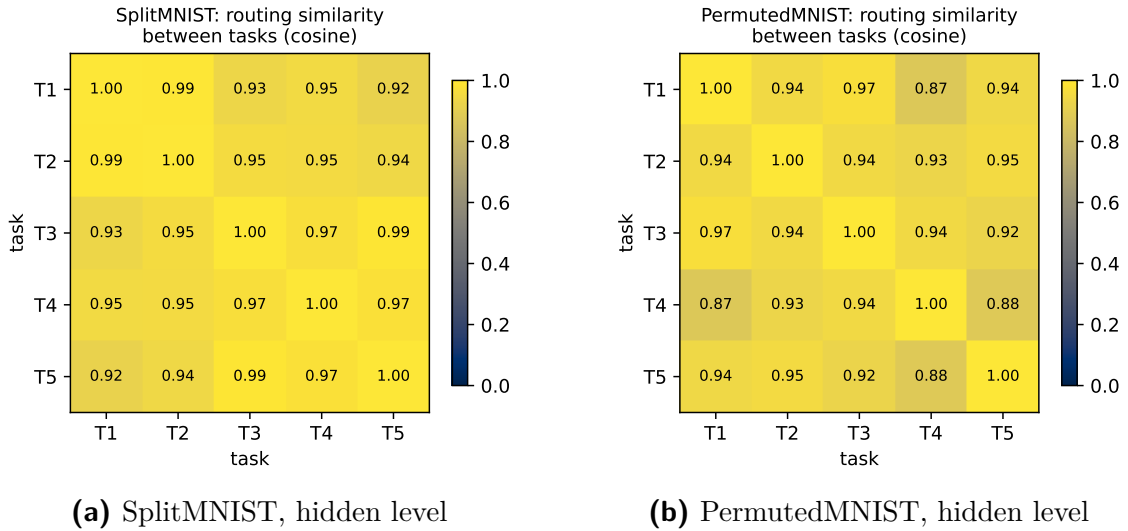


Figure 7.9.: Cosine similarity between the per task routing distributions. Off diagonal values close to 1 mean that two tasks use the experts in nearly the same way, which indicates a high interference risk; smaller values indicate task specific routing.

On SplitMNIST, the router develops a differentiated assignment: tasks 1 and 2 prefer the experts E3, E6 and E7, while later tasks distribute their probability mass more broadly and increasingly use E4 and E8, which were almost unused at the beginning. The routing is far from a perfectly disjoint partition, because E3 and E6 are shared by all tasks, which matches the observation that forgetting is reduced but not eliminated on this benchmark.

On PermutedMNIST with the hidden level model, the picture is completely different: every task routes most of its probability mass to the same two experts E2 and E5, with up to 0.32 each, and the pairwise routing similarities in Figure 7.9 are correspondingly close to one. With all tasks writing to the same experts, the interference is maximal. This is the direct, measurable cause of the poor result of the main setting on this benchmark.

The input level variant finally shows the desired behaviour on PermutedMNIST as well (Figure 7.8): the tasks occupy clearly different expert subsets, so the gradient updates of a new task mainly rewrite experts that carry little old knowledge. The router analysis thereby confirms the central hypothesis of the project at the mechanism level, and it also explains under which conditions the mechanism can work at all.

7.7. Per Task Analysis

The summary metrics compress the behaviour of a run into two numbers. Table 7.6. resolves the final accuracy per task and gives a more detailed picture of where the approaches differ.

Table 7.6.: Final test accuracy per task after the complete task sequence (mean over three seeds).

Benchmark	Approach	T1	T2	T3	T4	T5
SplitMNIST	Naive MLP	0.8657	0.8015	0.9742	0.9950	0.9918
	MLP + EWC	0.9398	0.8516	0.9861	0.9935	0.9928
	Wide MLP (naive)	0.8392	0.7671	0.9699	0.9946	0.9965
	MoE	0.9286	0.8115	0.9692	0.9847	0.9933
PermutedMNIST	Naive MLP	0.6235	0.6686	0.8329	0.9143	0.9786
	MLP + EWC	0.8803	0.8976	0.9406	0.9639	0.9767
	Wide MLP (naive)	0.7714	0.8754	0.9400	0.9581	0.9807
	MoE	0.5655	0.5250	0.7129	0.8746	0.9757

On SplitMNIST, all approaches keep the three later tasks almost perfectly; the differences are concentrated in the first two tasks, which were trained earliest and had the most time to be overwritten. Task 2 is the critical one: the naive MLP drops to 0.80, the wide MLP even to 0.77, while the MoE model holds 0.81 and EWC holds 0.85. The input level variant, which is not listed separately in the table, reaches the highest retention of all. A plausible reason for the special role of task 2 is its position in the sequence: it is early enough to be overwritten by three later tasks, but its two digit classes are visually more diverse than the classes 0 and 1 of the first task, so it is harder to relearn from shared features.

On PermutedMNIST, the pattern is much more regular. Every approach shows a monotone gradient: the older a task, the lower its final accuracy. This is the expected signature of a shared representation that is gradually overwritten. The gradient is steepest for the hidden level MoE, whose first task falls to 0.57, and flattest for EWC, which still reaches 0.88 on the first task after the complete sequence. The wide baseline lies clearly above the naive baseline on the early tasks, which confirms again that raw capacity helps on this benchmark, while the hidden level MoE lies below both, which confirms that its failure is structural and not a capacity problem.

The per task view also shows why the average forgetting of the MoE model on PermutedMNIST is so large: the model does not lose all tasks equally but sacrifices exactly the oldest ones, which is consistent with the router profile in Figure 7.7, where all tasks compete for the same few experts and the latest task wins.

7.8. Discussion

Taken together, the experiments answer the research question of this project in a differentiated way.

Does MoE reduce forgetting? Yes, when the routing is allowed to specialise and when the router sits at the right position in the architecture. On SplitMNIST, the hidden level MoE clearly outperforms both the naive and the capacity matched wide baseline. On PermutedMNIST, the input level MoE recovers a large part of the lost retention and its router separates the tasks again, although it stays slightly below the naive baseline in average accuracy. The wide baseline provides the control that isolates the mechanism: raw capacity helps only where the tasks share the input distribution, while routing helps where the router can separate the tasks.

What is the role of the router? The router acts as a learned, unsupervised task assignment module. Its profiles show that task specialisation emerges from ordinary training, and they also show the two conditions under which it fails: a too strong load balancing pressure, which forces uniform usage per task, and a shared trunk in front of the router, which mixes the very information that distinguishes the tasks. Both conditions could be identified only because the router behaviour is directly inspectable, which is itself an advantage of the MoE approach over implicit mechanisms.

How does MoE compare to EWC? EWC is a strong and robust method on both benchmarks and stays attractive because it does not change the architecture. The best MoE variant reaches or exceeds the EWC retention level on SplitMNIST, while on PermutedMNIST EWC remains ahead. The two approaches are, however, complementary rather than competing: a routed architecture combined with a light regularisation on the shared components or on the gate itself is a promising combination for future work.

Threats to validity. The study is limited to two MNIST based benchmarks with five tasks each and moderate network sizes; three seeds give a first estimate of the run to run variability but not a tight statistical bound. The results should therefore be read as a controlled proof of concept rather than as evidence for arbitrarily hard continual learning problems. Within these limits, however, every claim made in this chapter is backed by a measured and reproducible experiment.

8. Summary and Outlook

8.1. Achieved Results

The goal of this project was to find out whether a Mixture of Experts model can learn new data step by step without losing the knowledge it acquired earlier, and how the router of such a model contributes to this. All five goals that were fixed in Section 1.3 were reached.

First, the theoretical foundation was worked out. Catastrophic forgetting happens because every parameter of a dense network takes part in every task, so the gradient updates for a new task overwrite the solutions of the earlier ones. The Mixture of Experts architecture attacks exactly this root cause by routing every input through only a subset of its parameters.

Second, a Mixture of Experts model with a trainable router was designed and implemented, together with two MNIST based benchmarks and three comparison approaches, all in a clean and fully reproducible experimental framework.

Third and fourth, the model was trained sequentially on SplitMNIST and PermutedMNIST and compared quantitatively with a naive baseline, a capacity matched wide baseline and EWC. On SplitMNIST, the MoE model forgets clearly less than both baselines. On PermutedMNIST, the first architecture failed, and the detailed analysis of this failure became one of the most valuable results of the project: a too strong load balancing loss forces all tasks to share all experts, and the shared trunk in front of the router stores exactly the task specific information of this benchmark. An architecture variant with routing directly at the input, which was derived from this analysis, reaches the best retention of all compared approaches on SplitMNIST and recovers a large part of the lost retention on PermutedMNIST.

Fifth, the router analysis showed at the mechanism level how the protection works. Different tasks are routed to different expert subsets, so the gradient updates of a new task mainly rewrite parameters that carry little old knowledge. The analysis also showed under which conditions this mechanism fails, namely when the router cannot see the information that distinguishes the tasks.

Beyond the numbers, the project gave me exactly the practical experience that was hoped for in the proposal: from the theory of catastrophic forgetting, through the design and implementation of a non trivial architecture, to the disciplined experimental evaluation of a research hypothesis, including an unexpected negative result and its resolution.

8.2. Outlook

The results of this work can be extended and transferred in several ways.

8.2.1. Extensibility of the Results

On the architectural side, the model could grow new experts when the router saturates, which corresponds to the nice to have requirement N2 of Section 3.3 and would combine the MoE routing with the idea of progressive networks while keeping the growth sublinear. A gate consistency regulariser, which punishes changes of the routing distribution on new data, could stabilise the task to expert assignment further. The load balancing coefficient, which turned out to be a continual learning hyperparameter in this project, could be scheduled over time, for example strong at the beginning of a task and weak afterwards.

A second direction concerns the combination of mechanisms. The evaluation showed that EWC and MoE protect knowledge in complementary ways: EWC protects important weights wherever they are, while routing separates tasks structurally. A routed architecture with a light quadratic penalty on the shared components, especially on the trunk and the gate itself, is therefore a natural next step. The input level variant also leaves room for improvement on PermutedMNIST: its shared hidden layer behind the experts still mixes all permutations, so a second routed stage could close the remaining gap to the naive baseline.

8.2.2. Transferability of the Results

On the evaluation side, the approach should be transferred to convolutional backbones and harder benchmarks such as Split CIFAR-10 or Split FashionMNIST, and it should be tested in the class incremental scenario, where the router could additionally serve as a learned task inference module. The comparison should also be extended to replay based methods, which were deliberately excluded here. Finally, in the context of very large pre-trained models, where sparsely gated MoE layers are already standard, the present results add a small but concrete piece of evidence that expert routing is not only a capacity scaling device but also a promising mechanism for lifelong learning, provided the routing is placed where the tasks actually differ.

Bibliography

- [1] Michael McCloskey and Neal J. Cohen. “Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem”. In: *Psychology of Learning and Motivation* 24 (1989), pp. 109–165.
- [2] Robert M. French. “Catastrophic Forgetting in Connectionist Networks”. In: *Trends in Cognitive Sciences* 3.4 (1999), pp. 128–135.
- [3] Matthias De Lange et al. “A Continual Learning Survey: Defying Forgetting in Classification Tasks”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.7 (2021), pp. 3366–3385.
- [4] Liyuan Wang et al. “A Comprehensive Survey of Continual Learning: Theory, Method and Application”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.8 (2024), pp. 5362–5383.
- [5] James Kirkpatrick et al. “Overcoming Catastrophic Forgetting in Neural Networks”. In: *Proceedings of the National Academy of Sciences* 114.13 (2017), pp. 3521–3526.
- [6] Sylvestre-Alvise Rebuffi et al. “iCaRL: Incremental Classifier and Representation Learning”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.
- [7] David Lopez-Paz and Marc’Aurelio Ranzato. “Gradient Episodic Memory for Continual Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.
- [8] Andrei A. Rusu et al. “Progressive Neural Networks”. In: *arXiv preprint arXiv:1606.04671*. 2016.
- [9] Robert A. Jacobs et al. “Adaptive Mixtures of Local Experts”. In: *Neural Computation*. Vol. 3. 1. 1991, pp. 79–87.
- [10] Noam Shazeer et al. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *International Conference on Learning Representations (ICLR)*. 2017.
- [11] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: *Journal of Machine Learning Research* 23.120 (2022), pp. 1–39.

- [12] Martial Mermillod, Aurelia Bugaiska, and Patrick Bonin. “The Stability-Plasticity Dilemma: Investigating the Continuum from Catastrophic Forgetting to Age-Limited Learning Effects”. In: *Frontiers in Psychology*. Vol. 4. 2013, p. 504.
- [13] Yann LeCun et al. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE*. Vol. 86. 11. 1998, pp. 2278–2324.
- [14] Gido M. van de Ven and Andreas S. Tolias. “Three Scenarios for Continual Learning”. In: *arXiv preprint arXiv:1904.07734* (2019).
- [15] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [16] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015.
- [17] Ian J. Goodfellow et al. “An Empirical Investigation of Catastrophic Forgetting in Gradient-Based Neural Networks”. In: *arXiv preprint arXiv:1312.6211*. 2013.
- [18] Michael I. Jordan and Robert A. Jacobs. “Hierarchical Mixtures of Experts and the EM Algorithm”. In: *Neural Computation* 6.2 (1994), pp. 181–214.
- [19] Marc Masana et al. “Class-Incremental Learning: Survey and Performance Evaluation on Image Classification”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.5 (2022), pp. 5513–5533.
- [20] Friedemann Zenke, Ben Poole, and Surya Ganguli. “Continual Learning Through Synaptic Intelligence”. In: *International Conference on Machine Learning (ICML)*. 2017.
- [21] Rahaf Aljundi et al. “Memory Aware Synapses: Learning What (Not) to Forget”. In: *European Conference on Computer Vision (ECCV)*. 2018.
- [22] Zhizhong Li and Derek Hoiem. “Learning without Forgetting”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*. Vol. 40. 12. 2017, pp. 2935–2947.
- [23] Hanul Shin et al. “Continual Learning with Deep Generative Replay”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.
- [24] Arun Mallya and Svetlana Lazebnik. “PackNet: Adding Multiple Tasks to a Single Network by Iterative Pruning”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018.
- [25] Rahaf Aljundi, Punarjay Chakravarty, and Tinne Tuytelaars. “Expert Gate: Lifelong Learning with a Network of Experts”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.

- [26] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019.

A. Source Code

This appendix prints the complete source code of the core modules. The experiment drivers and the scripts for figures and tables follow the same style and are part of the submitted project folder together with all raw result files.

A.1. Benchmark Construction (datasets.py)

```
1  """Continual learning benchmark datasets.
2
3  Provides two standard benchmarks built on top of MNIST:
4
5  * SplitMNIST: the ten digit classes are divided into five consecutive
6    tasks with two classes each (0/1, 2/3, ..., 8/9).
7  * PermutedMNIST: every task uses all ten classes, but a different
8    fixed
9    random permutation is applied to the 784 input pixels.
10
11 Both benchmarks yield a sequence of (train, test) DataLoader pairs,
12   one per
13   task, which is exactly the setting in which catastrophic forgetting
14   occurs.
15 """
16
17 import os
18
19 import numpy as np
20 import torch
21 from torch.utils.data import DataLoader, Subset, TensorDataset
22 from torchvision import datasets, transforms
23
24 DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)),
25   ..", "data")
26
27 def _load_mnist(data_dir=DATA_DIR):
28     """Loads the raw MNIST train/test splits as tensors."""
29     tf = transforms.Compose([transforms.ToTensor()])
30     train = datasets.MNIST(data_dir, train=True, download=False,
31       transform=tf)
32     test = datasets.MNIST(data_dir, train=False, download=False,
33       transform=tf)
34     return train, test
```

```

30
31
32 def _flatten(dataset):
33     """Converts an MNIST dataset into flat (x, y) float tensors."""
34     xs = torch.stack([dataset[i][0] for i in range(len(dataset))])
35     ys = torch.tensor([dataset[i][1] for i in range(len(dataset))])
36     return xs.view(len(dataset), -1).float(), ys.long()
37
38
39 def make_split_mnist(n_tasks=5, batch_size=256, data_dir=DATA_DIR):
40     """Builds the SplitMNIST benchmark.
41
42     Task t contains the digits {2t, 2t+1}. Each task keeps its own
43     two-way
44     output head (task-incremental protocol).
45     """
46     train, test = _load_mnist(data_dir)
47     xtr, ytr = _flatten(train)
48     xte, yte = _flatten(test)
49
50     tasks = []
51     for t in range(n_tasks):
52         classes = [2 * t, 2 * t + 1]
53         tr_idx = [i for i in range(len(ytr)) if int(ytr[i]) in classes]
54         te_idx = [i for i in range(len(yte)) if int(yte[i]) in classes]
55
56         # Remap global labels (2t, 2t+1) to local head labels (0, 1).
57         ytr_local = (ytr[tr_idx] - 2 * t)
58         yte_local = (yte[te_idx] - 2 * t)
59         tr = TensorDataset(xtr[tr_idx], ytr_local)
60         te = TensorDataset(xte[te_idx], yte_local)
61         tasks.append((
62             DataLoader(tr, batch_size=batch_size, shuffle=True,
63                       drop_last=True),
64             DataLoader(te, batch_size=1024, shuffle=False),
65             len(classes),
66         ))
67     return tasks
68
69
70 class _PermutedDataset:
71     """Memory-efficient PermutedMNIST task dataset.
72
73     Instead of materialising a permuted copy of the whole data set for
74     every

```

```

71     task (about 190 MB per task), the permutation is applied on the
72     fly in
73     ‘‘__getitem__’’; the base tensors are stored only once and shared.
74     """
75
76     def __init__(self, x, y, perm):
77         self.x = x
78         self.y = y
79         self.perm = perm
80
81     def __len__(self):
82         return len(self.y)
83
84     def __getitem__(self, idx):
85         return self.x[idx][self.perm], self.y[idx]
86
87 def make_permuted_mnist(n_tasks=5, batch_size=256, seed=0, data_dir=
DATA_DIR):
88     """Builds the PermutedMNIST benchmark.
89
90     Every task applies its own fixed random pixel permutation; all ten
91     output
92     classes are shared, so a single ten-way head is used.
93     """
94
95     train, test = _load_mnist(data_dir)
96     xtr, ytr = _flatten(train)
97     xte, yte = _flatten(test)
98
99     rng = np.random.RandomState(seed)
100     tasks = []
101     for t in range(n_tasks):
102         if t == 0:
103             perm = np.arange(784)           # first task = original
104             MNIST
105         else:
106             perm = rng.permutation(784)
107             perm = torch.from_numpy(perm).long()
108             tr = _PermutedDataset(xtr, ytr, perm)
109             te = _PermutedDataset(xte, yte, perm)
110             tasks.append((
111                 DataLoader(tr, batch_size=batch_size, shuffle=True,
112                             drop_last=True),
113                 DataLoader(te, batch_size=1024, shuffle=False),
114                 10,
115             ))
116     return tasks

```

A.2. Models (models.py)

```

1  """Model definitions for the continual learning experiments.
2
3  Three families of models are defined:
4
5  * 'BaselineMLP': a plain multi-layer perceptron (naive fine-tuning
6    and
7    EWC baselines use this architecture).
8  * 'WideMLP': a wider MLP whose parameter count roughly matches the
9    MoE
10   model, used to separate the effect of additional capacity from the
11   effect of the MoE structure.
12 * 'MoENet': a Mixture of Experts network with a shared trunk
13   followed
14   by a MoE layer with a top-k gating ("router") network.
15
16 All models support multiple output heads so that SplitMNIST can be
17 evaluated
18 in the task-incremental protocol.
19 """
20
21 import torch
22 import torch.nn as nn
23 import torch.nn.functional as F
24
25 class MultiHeadMixin:
26     """Adds lazily created per-task output heads to a network."""
27
28     def _init_heads(self):
29         self.heads = nn.ModuleDict()
30
31     def _head_for(self, task_id, n_classes, device):
32         key = str(task_id)
33         if key not in self.heads:
34             head = nn.Linear(self.feature_dim, n_classes).to(device)
35             self.heads[key] = head
36         return self.heads[key]
37
38 class BaselineMLP(nn.Module, MultiHeadMixin):
39     """Plain two-hidden-layer perceptron used as the naive baseline.
40     """
41
42     def __init__(self, input_dim=784, hidden_dim=400):

```

```

40     super().__init__()
41     self.feature_dim = hidden_dim
42     self.fc1 = nn.Linear(input_dim, hidden_dim)
43     self.fc2 = nn.Linear(hidden_dim, hidden_dim)
44     self._init_heads()
45
46     def features(self, x):
47         h = F.relu(self.fc1(x))
48         return F.relu(self.fc2(h))
49
50     def forward(self, x, task_id=0, n_classes=10):
51         head = self._head_for(task_id, n_classes, x.device)
52         return head(self.features(x))
53
54
55 class WideMLP(nn.Module, MultiHeadMixin):
56     """Wider MLP with roughly as many parameters as the MoE model."""
57
58     def __init__(self, input_dim=784, hidden_dim=1100):
59         super().__init__()
60         self.feature_dim = hidden_dim
61         self.fc1 = nn.Linear(input_dim, hidden_dim)
62         self.fc2 = nn.Linear(hidden_dim, hidden_dim)
63         self._init_heads()
64
65     def features(self, x):
66         h = F.relu(self.fc1(x))
67         return F.relu(self.fc2(h))
68
69     def forward(self, x, task_id=0, n_classes=10):
70         head = self._head_for(task_id, n_classes, x.device)
71         return head(self.features(x))
72
73
74 class MoELayer(nn.Module):
75     """Mixture-of-Experts layer with top-k gating.
76
77     The *router* (gating network) produces a probability distribution
78     over
79     the ‘‘n_experts’’ experts for every input sample. Only the ‘‘
80     top_k’’
81     experts with the highest gate values are evaluated and their
82     outputs are
83     combined with the (re-normalised) gate values as weights. An
84     auxiliary
85     load-balancing loss encourages the router to use all experts
86     uniformly

```

```

82     over a batch, which prevents the router from collapsing onto a
83     single
84     expert early in training.
85     """
86
87     def __init__(self, dim, n_experts=8, top_k=2):
88         super().__init__()
89         self.n_experts = n_experts
90         self.top_k = top_k
91         self.experts = nn.ModuleList(
92             [nn.Sequential(nn.Linear(dim, dim), nn.ReLU()) for _ in
93              range(n_experts)]
94         )
95         self.router = nn.Linear(dim, n_experts)
96
97     def forward(self, x):
98         """Returns (mixed expert output, full gate probability matrix)
99         . """
100
101         logits = self.router(x) # [B, E]
102         gates = F.softmax(logits, dim=-1) # [B, E]
103         topv, topi = gates.topk(self.top_k, dim=-1) # [B, k]
104         topv = topv / topv.sum(dim=-1, keepdim=True) # re-normalise
105
106         out = torch.zeros_like(x)
107         for slot in range(self.top_k):
108             idx = topi[:, slot] # expert id per
109             # sample
110             w = topv[:, slot].unsqueeze(1) # mixing weight
111             for e in range(self.n_experts):
112                 mask = idx == e
113                 if mask.any():
114                     out[mask] += w[mask] * self.experts[e](x[mask])
115         return out, gates
116
117     def load_balance_loss(self, gates):
118         """Switch-style auxiliary loss: n_experts * sum(f_e * P_e).
119
120         f_e = fraction of samples routed to expert e (top-1 assignment
121         ),
122         P_e = mean gate probability of expert e over the batch.
123         """
124
125         f = torch.zeros(self.n_experts, device=gates.device)
126         top1 = gates.argmax(dim=-1)
127         for e in range(self.n_experts):
128             f[e] = (top1 == e).float().mean()
129         P = gates.mean(dim=0)
130         return self.n_experts * (f * P).sum()

```

```

124
125
126 class MoENet(nn.Module, MultiHeadMixin):
127     """MoE network: shared trunk, one MoE layer, per-task output heads
128         . """
129
130     def __init__(self, input_dim=784, hidden_dim=400, n_experts=8,
131                 top_k=2):
132         super().__init__()
133         self.feature_dim = hidden_dim
134         self.trunk = nn.Linear(input_dim, hidden_dim)
135         self.moe = MoELayer(hidden_dim, n_experts=n_experts, top_k=
136                             top_k)
137         self._init_heads()
138
139     def features(self, x):
140         h = F.relu(self.trunk(x))
141         h, gates = self.moe(h)
142         return h, gates
143
144     def forward(self, x, task_id=0, n_classes=10, return_gates=False):
145         head = self._head_for(task_id, n_classes, x.device)
146         h, gates = self.features(x)
147         logits = head(h)
148         if return_gates:
149             return logits, gates
150         return logits
151
152 class MoEInputLayer(nn.Module):
153     """MoE layer that routes the *raw input* (not a hidden
154         representation).
155
156     Identical gating mechanics to ‘MoELayer’, but the router and the
157     experts operate directly on the input features. Designed for
158     domain-incremental benchmarks such as PermutedMNIST, where the
159     task-specific information lives in the input mapping itself and a
160     shared input trunk would otherwise be overwritten by every new
161     task.
162     """
163
164     def __init__(self, in_dim=784, out_dim=256, n_experts=8, top_k=2):
165         super().__init__()
166         self.n_experts = n_experts
167         self.top_k = top_k
168         self.experts = nn.ModuleList(
169             [nn.Sequential(nn.Linear(in_dim, out_dim), nn.ReLU())

```

```

166         for _ in range(n_experts)]
167     )
168     self.router = nn.Linear(in_dim, n_experts)
169     self.out_dim = out_dim
170
171     def forward(self, x):
172         """Returns (mixed expert output, full gate probability matrix)
173         . """
174         logits = self.router(x)
175         gates = F.softmax(logits, dim=-1)
176         topv, topi = gates.topk(self.top_k, dim=-1)
177         topv = topv / topv.sum(dim=-1, keepdim=True)
178
179         out = torch.zeros(x.size(0), self.out_dim, device=x.device)
180         for slot in range(self.top_k):
181             idx = topi[:, slot]
182             w = topv[:, slot].unsqueeze(1)
183             for e in range(self.n_experts):
184                 mask = idx == e
185                 if mask.any():
186                     out[mask] += w[mask] * self.experts[e](x[mask])
187         return out, gates
188
189     def load_balance_loss(self, gates):
190         """Same Switch-style auxiliary loss as in 'MoELayer'."""
191         f = torch.zeros(self.n_experts, device=gates.device)
192         top1 = gates.argmax(dim=-1)
193         for e in range(self.n_experts):
194             f[e] = (top1 == e).float().mean()
195         P = gates.mean(dim=0)
196         return self.n_experts * (f * P).sum()
197
198     class MoEInputNet(nn.Module, MultiHeadMixin):
199         """MoE variant with input-level routing (ablation/comparison model)
200         ).
201
202         Structure: input-level MoE (8 experts, top-2) -> shared hidden
203         layer ->
204         per-task heads. Roughly matches the parameter count of 'MoENet
205         '."""
206
207         def __init__(self, input_dim=784, expert_dim=256, hidden_dim=256,
208             n_experts=8, top_k=2):
209             super().__init__()
210             self.feature_dim = hidden_dim

```

```

209         self.moe = MoEInputLayer(input_dim, expert_dim, n_experts,
210                                   top_k)
211         self.fc = nn.Linear(expert_dim, hidden_dim)
212         self._init_heads()
213
214     def forward(self, x, task_id=0, n_classes=10, return_gates=False):
215         head = self._head_for(task_id, n_classes, x.device)
216         h, gates = self.moe(x)
217         h = F.relu(self.fc(h))
218         logits = head(h)
219         if return_gates:
220             return logits, gates
221         return logits
222
223 def count_parameters(model):
224     """Total number of trainable parameters of a model."""
225     return sum(p.numel() for p in model.parameters() if p.
                requires_grad)

```

A.3. Strategies and Metrics (continual.py)

```

1  """Continual learning training strategies and evaluation metrics.
2
3  Strategies:
4
5  * 'NaiveTrainer': trains sequentially on one task after another
6    without
7    any protection against forgetting (standard fine tuning).
8  * 'EWCTrainer': Elastic Weight Consolidation. After finishing a task
9    ,
10   the diagonal Fisher information of the parameters is estimated and a
11   quadratic penalty pulls important weights back towards their old
12   values
13   while learning later tasks.
14  * 'MoETrainer': trains a MoENet sequentially with an additional
15    load balancing loss on the router.
16
17  Metrics:
18  The accuracy matrix R (entry  $R[i, j]$  = accuracy on task  $j$  after
19    training on
20    task  $i$ ) is the basis for average accuracy, average forgetting and
21    backward
22    transfer (BWT).
23  """

```

```

19
20 import numpy as np
21 import torch
22 import torch.nn.functional as F
23
24
25 # Evaluation helpers
26
27 @torch.no_grad()
28 def evaluate(model, loader, task_id, n_classes, device):
29     """Accuracy of a model on one task's test set."""
30     model.eval()
31     correct, total = 0, 0
32     for x, y in loader:
33         x, y = x.to(device), y.to(device)
34         logits = model(x, task_id=task_id, n_classes=n_classes)
35         correct += int((logits.argmax(dim=-1) == y).sum())
36         total += len(y)
37     return correct / total
38
39
40 def accuracy_matrix_at_end(accs):
41     """Computes the standard continual learning metrics.
42
43     ‘accs’ is a list of lists; accs[i][j] is the accuracy on task j
44     after
45     training task i (j <= i).
46     """
47     n = len(accs)
48     R = np.full((n, n), np.nan)
49     for i in range(n):
50         for j in range(i + 1):
51             R[i, j] = accs[i][j]
52
53     avg_acc = float(np.nanmean(R[-1, :]))
54     # Forgetting of task j: best previous accuracy minus final
55     accuracy.
56     forgetting = []
57     for j in range(n - 1):
58         best = np.nanmax(R[:-1, j])
59         forgetting.append(best - R[-1, j])
60     avg_forgetting = float(np.mean(forgetting))
61     bwt = float(np.nanmean(R[-1, :n - 1] - np.diag(R)[:n - 1]))
62     return R, avg_acc, avg_forgetting, bwt
63
64 # Trainers

```

```

64
65 class NaiveTrainer:
66     """Sequential fine-tuning without any continual learning mechanism
        ."""
67
68     name = "Naive MLP"
69
70     def __init__(self, model, lr=1e-3, epochs=5, device="cpu"):
71         self.model = model.to(device)
72         self.lr = lr
73         self.epochs = epochs
74         self.device = device
75         self.accs = [] # per-stage accuracy rows
76
77     def _loss(self, logits, y):
78         return F.cross_entropy(logits, y)
79
80     def train_task(self, task_id, train_loader, n_classes):
81         self.model.train()
82         opt = torch.optim.Adam(self.model.parameters(), lr=self.lr)
83         for _ in range(self.epochs):
84             for x, y in train_loader:
85                 x, y = x.to(self.device), y.to(self.device)
86                 logits = self.model(x, task_id=task_id, n_classes=
                    n_classes)
87                 loss = self._loss(logits, y)
88                 opt.zero_grad()
89                 loss.backward()
90                 opt.step()
91
92     def run(self, tasks):
93         """Trains on all tasks sequentially, evaluating after every
            stage."""
94         for t, (tr, _, n_classes) in enumerate(tasks):
95             self.train_task(t, tr, n_classes)
96             row = [evaluate(self.model, tasks[j][1], j, tasks[j][2],
                self.device)
                for j in range(t + 1)]
97             self.accs.append(row)
98         return self.accs
99
100
101
102 class EWCTrainer(NaiveTrainer):
103     """Elastic Weight Consolidation (Kirkpatrick et al., 2017)."""
104
105     name = "MLP + EWC"
106

```

```

107     def __init__(self, model, lr=1e-3, epochs=5, device="cpu",
108                 ewc_lambda=500.0):
109         super().__init__(model, lr, epochs, device)
110         self.ewc_lambda = ewc_lambda
111         self._fisher = None      # diagonal Fisher per parameter
112                                   task
113
114     def _loss(self, logits, y):
115         loss = F.cross_entropy(logits, y)
116         if self._fisher is not None:
117             penalty = 0.0
118             for p, f, s in zip(self.model.parameters(), self._fisher,
119                               self._star):
120                 penalty = penalty + (f * (p - s) ** 2).sum()
121             loss = loss + 0.5 * self.ewc_lambda * penalty
122         return loss
123
124     @torch.no_grad()
125     def _copy_params(self):
126         return [p.clone() for p in self.model.parameters()]
127
128     def _estimate_fisher(self, loader, task_id, n_classes, n_batches
129                        =50):
130         """Empirical diagonal Fisher over (at most) ``n_batches``
131            batches."""
132         fisher = [torch.zeros_like(p) for p in self.model.parameters()]
133         self.model.eval()
134         for i, (x, y) in enumerate(loader):
135             if i >= n_batches:
136                 break
137             x, y = x.to(self.device), y.to(self.device)
138             self.model.zero_grad()
139             logits = self.model(x, task_id=task_id, n_classes=
140                               n_classes)
141             F.cross_entropy(logits, y).backward()
142             for f, p in zip(fisher, self.model.parameters()):
143                 # Heads of other tasks do not participate in this task
144                 # computation graph, so their gradient stays None.
145                 if p.grad is not None:
146                     f += p.grad.detach() ** 2
147         fisher = [f / min(n_batches, len(loader)) for f in fisher]
148         if self._fisher is None:
149             self._fisher = fisher
150         else: # accumulate penalties over tasks (standard practice)

```

```

146         self._fisher = [fold + fnew for fold, fnew in zip(self.
147             _fisher, fisher)]
148
149     def train_task(self, task_id, train_loader, n_classes):
150         super().train_task(task_id, train_loader, n_classes)
151         self._estimate_fisher(train_loader, task_id, n_classes)
152         self._star = self._copy_params()
153
154     class MoETrainer(NaiveTrainer):
155         """Sequential training of a MoENet with a load-balancing auxiliary
156             loss."""
157
158         name = "MoE"
159
160         def __init__(self, model, lr=1e-3, epochs=5, device="cpu",
161             lb_coeff=0.01):
162             super().__init__(model, lr, epochs, device)
163             self.lb_coeff = lb_coeff
164
165         def train_task(self, task_id, train_loader, n_classes):
166             self.model.train()
167             opt = torch.optim.Adam(self.model.parameters(), lr=self.lr)
168             for _ in range(self.epochs):
169                 for x, y in train_loader:
170                     x, y = x.to(self.device), y.to(self.device)
171                     logits, gates = self.model(x, task_id=task_id,
172                         n_classes=n_classes,
173                         return_gates=True)
174                     loss = (F.cross_entropy(logits, y)
175                         + self.lb_coeff * self.model.moe.
176                             load_balance_loss(gates))
177                     opt.zero_grad()
178                     loss.backward()
179                     opt.step()
180
181             @torch.no_grad()
182             def router_profile(self, tasks):
183                 """Mean gate probability of every expert for every task's test
184                     data.
185
186                 Returns a [n_tasks, n_experts] matrix, the central artefact
187                     for the
188                 router analysis in the report.
189                 """
190                 self.model.eval()
191                 n_experts = self.model.moe.n_experts

```

```
186     profile = np.zeros((len(tasks), n_experts))
187     for t, (_, te, n_classes) in enumerate(tasks):
188         total = np.zeros(n_experts)
189         count = 0
190         for x, _ in te:
191             x = x.to(self.device)
192             _, gates = self.model(x, task_id=t, n_classes=
193                             n_classes,
194                             return_gates=True)
195             total += gates.sum(dim=0).cpu().numpy()
196             count += len(x)
197         profile[t] = total / count
198     return profile
```

B. Experiment Driver and Logs

B.1. Experiment Driver (run_experiments.py)

```
1  """Main experiment driver.
2
3  Trains all model/strategy combinations sequentially on the SplitMNIST
   and
4  PermutedMNIST benchmarks and stores the accuracy matrices, the summary
5  metrics and the router profiles as JSON/NPZ files in '../results'.
6
7  Usage:  python3 run_experiments.py
8  """
9
10 import json
11 import os
12 import sys
13 import time
14
15 import numpy as np
16 import torch
17
18 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
19 from datasets import make_split_mnist, make_permuted_mnist
20 from models import BaselineMLP, WideMLP, MoENet, count_parameters
21 from continual import NaiveTrainer, EWCTrainer, MoETrainer,
   accuracy_matrix_at_end
22
23 RESULTS = "../results"
24 SEEDS = [0, 1, 2]
25 N_TASKS = 5
26 EPOCHS = 5
27 DEVICE = "cpu"
28
29
30 def build_trainers(seed):
31     """Creates one trainer per compared approach (identical seed for
   all)."""
32     torch.manual_seed(seed)
33     naive = NaiveTrainer(BaselineMLP(), epochs=EPOCHS, device=DEVICE)
34     torch.manual_seed(seed)
35     ewc = EWCTrainer(BaselineMLP(), epochs=EPOCHS, device=DEVICE,
   ewc_lambda=500.0)
36     torch.manual_seed(seed)
```

```

37     wide = NaiveTrainer(WideMLP(), epochs=EPOCHS, device=DEVICE)
38     wide.name = "Wide MLP (naive)"
39     torch.manual_seed(seed)
40     moe = MoETrainer(MoENet(n_experts=8, top_k=2), epochs=EPOCHS,
41                     device=DEVICE,
42                     lb_coeff=0.01)
43     return [naive, ewc, wide, moe]
44
45 def save_partial(results):
46     """Writes the results collected so far to disk (crash-safe)."""
47     with open(os.path.join(RESULTS, "results.json"), "w") as f:
48         json.dump(results, f, indent=2)
49
50
51 def run_benchmark(name, tasks_fn, results):
52     """Runs all approaches on one benchmark for all seeds."""
53     out = {"benchmark": name, "approaches": {}}
54     for seed in SEEDS:
55         tasks = tasks_fn(seed)
56         for trainer in build_trainers(seed):
57             t0 = time.time()
58             accs = trainer.run(tasks)
59             R, avg_acc, avg_forget, bwt = accuracy_matrix_at_end(accs)
60             rec = {
61                 "seed": seed,
62                 "R": R.tolist(),
63                 "avg_acc": avg_acc,
64                 "avg_forgetting": avg_forget,
65                 "bwt": bwt,
66                 "params": count_parameters(trainer.model),
67                 "runtime_s": round(time.time() - t0, 1),
68             }
69             if hasattr(trainer, "router_profile"):
70                 profile = trainer.router_profile(tasks)
71                 rec["router_profile"] = profile.tolist()
72                 np.savez(os.path.join(
73                     RESULTS, f"router_{name}_seed{seed}.npz"), profile
74                     =profile)
75             out["approaches"].setdefault(trainer.name, []).append(rec)
76             # Persist after every run so a crash cannot lose earlier
77             # runs.
78             save_partial(results + [out])
79             print(f"[{name}] seed={seed} {trainer.name}: "
80                   f"avg_acc={avg_acc:.4f} forgetting={avg_forget:.4f} "
81                   " "
82                   f"bwt={bwt:.4f} ({rec['runtime_s']}s)", flush=True)

```

```

80     return out
81
82
83 def main():
84     os.makedirs(RESULTS, exist_ok=True)
85     results = []
86
87     print("=== SplitMNIST ===", flush=True)
88     results.append(run_benchmark(
89         "split_mnist",
90         lambda seed: make_split_mnist(n_tasks=N_TASKS), results))
91
92     print("=== PermutedMNIST ===", flush=True)
93     results.append(run_benchmark(
94         "permuted_mnist",
95         lambda seed: make_permuted_mnist(n_tasks=N_TASKS, seed=seed),
96         results))
97
98     save_partial(results)
99     print("Saved results to results/results.json", flush=True)
100
101 if __name__ == "__main__":
102     main()

```

B.2. Experiment Log

The following listing shows the complete log of the main experiment suite. Every line documents one continual training run with its final metrics.

```

1  === SplitMNIST ===
2  [split_mnist] seed=0 Naive MLP: avg_acc=0.8845 forgetting=0.1414 bwt=-0.1414 (25.8s)
3  [split_mnist] seed=0 MLP + EWC: avg_acc=0.9605 forgetting=0.0448 bwt=-0.0448 (25.6s)
4  [split_mnist] seed=0 Wide MLP (naive): avg_acc=0.9308 forgetting=0.0841 bwt=-0.0841
   (65.1s)
5  [split_mnist] seed=0 MoE: avg_acc=0.9523 forgetting=0.0561 bwt=-0.0561 (67.5s)
6  [split_mnist] seed=1 Naive MLP: avg_acc=0.9517 forgetting=0.0558 bwt=-0.0557 (18.0s)
7  [split_mnist] seed=1 MLP + EWC: avg_acc=0.9181 forgetting=0.0984 bwt=-0.0984 (25.6s)
8  [split_mnist] seed=1 Wide MLP (naive): avg_acc=0.9241 forgetting=0.0924 bwt=-0.0924
   (64.0s)
9  [split_mnist] seed=1 MoE: avg_acc=0.9535 forgetting=0.0538 bwt=-0.0538 (67.3s)
10 [split_mnist] seed=2 Naive MLP: avg_acc=0.9407 forgetting=0.0697 bwt=-0.0697 (18.3s)
11 [split_mnist] seed=2 MLP + EWC: avg_acc=0.9796 forgetting=0.0220 bwt=-0.0220 (27.8s)
12 [split_mnist] seed=2 Wide MLP (naive): avg_acc=0.8855 forgetting=0.1406 bwt=-0.1406
   (64.1s)
13 [split_mnist] seed=2 MoE: avg_acc=0.9066 forgetting=0.1125 bwt=-0.1125 (68.0s)
14 === PermutedMNIST ===
15 [permuted_mnist] seed=0 Naive MLP: avg_acc=0.7888 forgetting=0.2358 bwt=-0.2358 (101.0
   s)

```

```
16 [permuted_mnist] seed=0 MLP + EWC: avg_acc=0.9321 forgetting=0.0566 bwt=-0.0566 (131.2
    s)
17 [permuted_mnist] seed=0 Wide MLP (naive): avg_acc=0.8882 forgetting=0.1163 bwt=-0.1163
    (341.5s)
18 [permuted_mnist] seed=0 MoE: avg_acc=0.7126 forgetting=0.3293 bwt=-0.3293 (1280.0s)
19 [permuted_mnist] seed=1 Naive MLP: avg_acc=0.8036 forgetting=0.2176 bwt=-0.2176 (101.1
    s)
20 [permuted_mnist] seed=1 MLP + EWC: avg_acc=0.9144 forgetting=0.0776 bwt=-0.0776 (128.3
    s)
21 [permuted_mnist] seed=1 Wide MLP (naive): avg_acc=0.9050 forgetting=0.0919 bwt=-0.0919
    (335.3s)
22 [permuted_mnist] seed=1 MoE: avg_acc=0.6979 forgetting=0.3474 bwt=-0.3474 (353.9s)
23 [permuted_mnist] seed=2 Naive MLP: avg_acc=0.8183 forgetting=0.2004 bwt=-0.2004 (100.3
    s)
24 [permuted_mnist] seed=2 MLP + EWC: avg_acc=0.9490 forgetting=0.0354 bwt=-0.0354 (128.8
    s)
25 [permuted_mnist] seed=2 Wide MLP (naive): avg_acc=0.9222 forgetting=0.0718 bwt=-0.0718
    (332.3s)
26 [permuted_mnist] seed=2 MoE: avg_acc=0.7817 forgetting=0.2424 bwt=-0.2424 (351.5s)
27 Saved results to results/results.json
```